En sql server que son los filegroup

En SQL Server, un **Filegroup** (grupo de archivos) es una unidad lógica de almacenamiento que agrupa uno o varios archivos físicos de datos (`.mdf` y `.ndf`) para facilitar la administración, el rendimiento y la distribución del almacenamiento de una base de datos.

¿Cómo funciona la estructura de archivos?

Una base de datos en SQL Server se compone de:

1. **Archivo de Datos Primario (`.mdf`):** Contiene la información de inicio de la base de datos y apunta a los demás archivos. Pertenece obligatoriamente al Filegroup primario.
2. **Archivos de Datos Secundarios (`.ndf`):** Archivos de datos adicionales. Se pueden asignar al Filegroup primario o a Filegroups secundarios personalizados.
3. **Archivo del Log de Transacciones (`.ldf`):** Registra las transacciones. **Los archivos de log NO forman parte de los Filegroups.**

```
Base de Datos
 ├── Log de Transacciones (.ldf)  [Fuera de Filegroups]
 └── Filegroups (Almacenamiento de Datos)
      ├── PRIMARY (Filegroup por defecto)
      │    └── Principal.mdf
      └── FG_Historico (Filegroup Secundario)
           ├── Datos2024_1.ndf
           └── Datos2024_2.ndf
```

Tipos de Filegroups

PRIMARY: Es el Filegroup por defecto que crea SQL Server. Contiene el archivo `.mdf` y las tablas del sistema. Si no creas otros Filegroups, todas las tablas e

índices de usuario se guardarán aquí.

Secundario (User-Defined): Creado explícitamente por el administrador del sistema para organizar datos específicos (por ejemplo, separar datos históricos, tablas de auditoría o índices).

FILESTREAM / Memory-Optimized: Filegroups especiales para almacenar datos no estructurados (imágenes, documentos en disco) o tablas optimizadas en memoria (In-Memory OLTP).

¿Para qué sirven y qué ventajas ofrecen?

1. **Mejora del Rendimiento de E/S (I/O):**
   Puedes crear un Filegroup secundario con varios archivos `.ndf` ubicados en discos físicos o volúmenes de almacenamiento distintos. SQL Server distribuirá la escritura entre ellos de forma proporcional (algoritmo *proportional fill*), aumentando la velocidad de lectura y escritura.
2. **Estrategias de Backup y Restauración Granulares:**
   En bases de datos muy grandes (VLDB), hacer un backup completo puede tardar horas. Con Filegroups puedes realizar copias de seguridad o restauraciones parciales. Por ejemplo, restaurar solo el Filegroup operacional mientras el Filegroup de datos históricos archivados permanece en modo solo lectura.
3. **Mantenimiento y Administración Facilitada:**
   Puedes marcar un Filegroup secundario como de **Solo Lectura** (`READONLY`). Esto optimiza las consultas sobre datos históricos (ya que SQL Server no necesita gestionar bloqueos de escritura) y reduce el tamaño de los backups frecuentes.
4. **Particionamiento de Tablas e Índices:**
   Permite dividir una tabla grande por rangos (por ejemplo, por años o meses) y colocar cada partición en un Filegroup diferente alojado en distintos tipos de almacenamiento (ej. SSDs rápidos para datos recientes, discos HDD lentos para años anteriores).

Ejemplo de uso en T-SQL

SQL

```sql
-- 1. Crear la base de datos con un Filegroup secundario
CREATE DATABASE [TiendaOnline]
ON PRIMARY
```

```
( NAME = N'Tienda_Primary', FILENAME = N'C:\SQLData\Tienda.mdf' ),
FILEGROUP [FG_Historico]
( NAME = N'Tienda_Hist1', FILENAME = N'D:\SQLData\Tienda_Hist1.ndf' )
LOG ON
( NAME = N'Tienda_Log', FILENAME = N'E:\SQLLogs\Tienda_log.ldf' );
GO

-- 2. Crear una tabla en el Filegroup secundario
CREATE TABLE [dbo].[VentasAntiguas] (
    IDInt INT PRIMARY KEY,
    Fecha DATETIME,
    Monto DECIMAL(18,2)
) ON [FG_Historico];
GO
```

actua como un experto en sql server. creame un ejemplo desde cero de como puedo crear una tabla particionada en sql server

Para crear una tabla particionada desde cero en SQL Server, necesitamos definir una **Partition Function** (que dicta *cómo* se dividen los datos según sus límites) y un **Partition Scheme** (que dicta *dónde* se guardan físicamente esas particiones dentro de los Filegroups).

Tomemos como ejemplo una tabla de **Ventas** particionada por fecha para separar los años **2023, 2024, 2025** y años futuros.

Paso 1: Crear los Filegroups y Archivos de Datos

Primero organizamos la estructura de almacenamiento. Crearemos un Filegroup y un archivo `.ndf` dedicado para cada año.

SQL

```
USE [master];
GO

-- 1. Crear la base de datos básica
CREATE DATABASE [DB_Particionada];
GO

USE [DB_Particionada];
GO

-- 2. Agregar los Filegroups
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_Anteriores];
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_2023];
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_2024];
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_2025];
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_Futuro];
GO

-- 3. Asignar un archivo de datos (.ndf) a cada Filegroup
-- (Asegúrate de que la ruta C:\SQLData\ exista o cámbiala por tu ruta local)
ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_Ant', FILENAME = N'C:\SQLData\Ventas_Ant.ndf' ) TO FILEGROUP [F

ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_2023', FILENAME = N'C:\SQLData\Ventas_2023.ndf' ) TO FILEGROUP

ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_2024', FILENAME = N'C:\SQLData\Ventas_2024.ndf' ) TO FILEGROUP

ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_2025', FILENAME = N'C:\SQLData\Ventas_2025.ndf' ) TO FILEGROUP

ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_Fut', FILENAME = N'C:\SQLData\Ventas_Fut.ndf' ) TO FILEGROUP [F
GO
```

Paso 2: Crear la Función de Partición (Partition Function)

La función define la columna pivote (en este caso de tipo `DATETIME2` o `DATE`) y establece las fronteras de corte. Al usar `RANGE RIGHT`, la fecha del límite cae en la partición de la derecha (siguiente rango).

SQL

```sql
CREATE PARTITION FUNCTION PF_Ventas_PorAnio (DATETIME2)
AS RANGE RIGHT FOR VALUES
(
    '2023-01-01 00:00:00', -- Partición 1: < 2023 | Partición 2: >= 2023
    '2024-01-01 00:00:00', -- Partición 3: >= 2024
    '2025-01-01 00:00:00', -- Partición 4: >= 2025
    '2026-01-01 00:00:00'  -- Partición 5: >= 2026
);
GO
```

Paso 3: Crear el Esquema de Partición (Partition Scheme)

El esquema mapea el resultado de la función de partición hacia los Filegroups que creamos en el Paso 1.

> **Nota:** *Como tenemos* **4 límites** *en la función de partición, se generan* **5 particiones en total.** *Por lo tanto, debemos pasarle 5 Filegroups al esquema.*

SQL

```sql
CREATE PARTITION SCHEME PS_Ventas_PorAnio
AS PARTITION PF_Ventas_PorAnio
TO
(
    [FG_Ventas_Anteriores], -- Para registros < 2023
    [FG_Ventas_2023],       -- Para 2023
    [FG_Ventas_2024],       -- Para 2024
    [FG_Ventas_2025],       -- Para 2025
    [FG_Ventas_Futuro]      -- Para 2026 en adelante
);
GO
```

Paso 4: Crear la Tabla en el Esquema de Partición

Al definir la tabla, la clave primaria **debe incluir obligatoriamente la columna de partición** (en este caso `FechaVenta`). Al final, en lugar del clásico `ON [PRIMARY]`, le indicamos que se cree `ON PS_Ventas_PorAnio(FechaVenta)`.

SQL

```sql
CREATE TABLE dbo.Ventas
(
    VentaID INT IDENTITY(1,1) NOT NULL,
    FechaVenta DATETIME2 NOT NULL,
    ClienteID INT NOT NULL,
    Monto DECIMAL(18,2) NOT NULL,
    CONSTRAINT PK_Ventas PRIMARY KEY CLUSTERED (VentaID, FechaVenta)
) ON PS_Ventas_PorAnio(FechaVenta);
GO
```

Paso 5: Probar la Inserción de Datos

Insertemos registros con distintas fechas para validar que SQL Server redirija cada fila a su correspondiente partición de forma transparente:

SQL

```sql
INSERT INTO dbo.Ventas (FechaVenta, ClienteID, Monto) VALUES
('2022-05-15', 101, 150.00), -- Caerá en FG_Ventas_Anteriores
('2023-03-20', 102, 200.50), -- Caerá en FG_Ventas_2023
('2024-07-10', 103, 99.99),  -- Caerá en FG_Ventas_2024
('2025-11-01', 104, 500.00), -- Caerá en FG_Ventas_2025
('2026-02-14', 105, 120.00); -- Caerá en FG_Ventas_Futuro
GO
```

Paso 6: Verificar en qué partición quedó cada registro

Puedes consultar las vistas de sistema de SQL Server para confirmar la distribución exacta de las filas en los Filegroups y ver el comportamiento del *Partition Elimination*:

SQL

```sql
SELECT
    p.partition_number AS [Número Partición],
    fg.name AS [Nombre Filegroup],
    p.rows AS [Total Filas],
    rv.value AS [Límite Inclusivo Derecha]
FROM sys.partitions p
INNER JOIN sys.indexes i
    ON p.object_id = i.object_id AND p.index_id = i.index_id
INNER JOIN sys.data_spaces ds
    ON i.data_space_id = ds.data_space_id
INNER JOIN sys.destination_data_spaces dds
    ON ds.data_space_id = dds.partition_scheme_id AND p.partition_number = dds.de
INNER JOIN sys.filegroups fg
    ON dds.data_space_id = fg.data_space_id
LEFT JOIN sys.partition_range_values rv
    ON rv.boundary_id = p.partition_number
WHERE p.object_id = OBJECT_ID('dbo.Ventas');
```

¿Quieres ver cómo automatizar la creación de nuevas particiones para años futuros mediante SPLIT?

↪ Sí

En SQL Server, ¿cómo puedo usar SPLIT y MERGE RANGE para agregar una nueva partición a una tabla particionada existente?

Para agregar o consolidar rangos de fechas en una tabla particionada existente se utilizan las sentencias `ALTER PARTITION FUNCTION ... SPLIT RANGE` (para dividir una partición existente en dos) y `MERGE RANGE` (para fusionar dos particiones contiguas en una).

1. Agregar una nueva partición (`SPLIT RANGE`)

Cuando deseas agregar un nuevo rango (por ejemplo, preparar el almacenamiento para el año **2027**), debes seguir dos pasos clave: indicar en el **Esquema de Partición** cuál será el nuevo Filegroup destino y luego ejecutar el `SPLIT` en la **Función de Partición**.

Paso 1.1: Asignar o crear el nuevo Filegroup

SQL

```sql
USE [DB_Particionada];
GO

-- 1. Crear el nuevo Filegroup y asignarle un archivo .ndf
ALTER DATABASE [DB_Particionada] ADD FILEGROUP [FG_Ventas_2026];
ALTER DATABASE [DB_Particionada] ADD FILE
( NAME = N'Ventas_2026', FILENAME = N'C:\SQLData\Ventas_2026.ndf' ) TO FILEGROUP
GO
```

Paso 1.2: Marcar el Filegroup en el Partition Scheme y hacer el SPLIT

SQL

```sql
-- 2. Marcar el nuevo Filegroup como NEXT USED en el esquema
ALTER PARTITION SCHEME PS_Ventas_PorAnio
NEXT USED [FG_Ventas_2026];
GO

-- 3. Dividir el rango en la función de partición
ALTER PARTITION FUNCTION PF_Ventas_PorAnio()
SPLIT RANGE ('2027-01-01 00:00:00');
GO
```

Efecto de esta operación:
SQL Server toma el Filegroup que tenía el límite superior (`FG_Ventas_Futuro` *) y reubica o asigna los datos que correspondan a* `>= 2027-01-01` *al nuevo Filegroup* `FG_Ventas_2026` *, dejando los datos* `>= 2026-01-01` *y* `< 2027-01-01` *correctamente delimitados.*

2. Fusionar o eliminar una partición antigua (`MERGE RANGE`)

Si necesitas consolidar datos viejos (por ejemplo, unir todos los datos anteriores a 2024 en un solo rango histórico), utilizas `MERGE RANGE` . **No necesitas tocar el Partition Scheme**, SQL Server elimina la frontera automáticamente.

SQL

```
-- Eliminar el límite '2023-01-01', fusionando la partición < 2023 con la partici
ALTER PARTITION FUNCTION PF_Ventas_PorAnio()
MERGE RANGE ('2023-01-01 00:00:00');
GO
```

Efecto de esta operación:
Las filas que estaban separadas por la frontera `'2023-01-01'` ahora pertenecen a un mismo rango continuo. Los datos se mantienen en el Filegroup persistente de la partición resultante.

3. Consideraciones de Rendimiento e Impacto (Best Practices)

1. **Evita mover filas en producción (Data Movement):**

   `SPLIT RANGE` : Ejecuta el `SPLIT` siempre en una partición que esté **completamente vacía** (por ejemplo, creando los cortes de años futuros antes de que lleguen los datos). Si haces un `SPLIT` sobre una partición con millones de registros, SQL Server tendrá que mover físicamente las filas de un archivo a otro, generando bloqueos extensos e incrementando el uso del Log de Transacciones.

   `MERGE RANGE` : Procura fusionar particiones cuando ambas pertenezcan al mismo Filegroup o cuando una de ellas no contenga filas.

2. **Verificar el estado tras las modificaciones:**
   Puedes ejecutar la siguiente consulta para comprobar cómo quedaron redistribuidas las particiones, sus límites y el número de filas por rango:

SQL

```
SELECT
    p.partition_number AS [N° Partición],
    fg.name AS [Filegroup],
    p.rows AS [Total Filas],
    rv.value AS [Límite]
FROM sys.partitions p
INNER JOIN sys.indexes i
    ON p.object_id = i.object_id AND p.index_id = i.index_id
INNER JOIN sys.data_spaces ds
    ON i.data_space_id = ds.data_space_id
INNER JOIN sys.destination_data_spaces dds
    ON ds.data_space_id = dds.partition_scheme_id AND p.partition_number = dds.de
INNER JOIN sys.filegroups fg
    ON dds.data_space_id = fg.data_space_id
LEFT JOIN sys.partition_range_values rv
    ON rv.boundary_id = p.partition_number
WHERE p.object_id = OBJECT_ID('dbo.Ventas');
```

¿Quieres aprender a realizar una purga instantánea de datos mediante SWITCH PARTITION?

↪ Sí

quiero comentarte algo, en hive una tabla tiene por lo general una particion por fecha, me explico una tabla recibe por ejemplo la carga de un archivoq ue tiene 100 registros , pero corresponden por ejemplo las ventas del dia

En SQL Server, la estrategia equivalente para lograr esto sin necesidad de recrear particiones manualmente cada día se basa en el **particionamiento por rangos dinámico** combinado con **vistas y particionamiento mensual/anual** o el uso de la técnica `SWITCH PARTITION`.

A diferencia de Apache Hive (que crea subdirectorios dinámicos `fecha=20261004` automáticamente al insertar datos con un valor de partición nuevo), SQL Server requiere

que el rango de la partición y su soporte de almacenamiento fijos existan previamente.

Existen **dos formas principales** de replicar esta arquitectura en SQL Server:

Enfoque 1: Particionamiento por Rangos Diarios (El homólogo directo a Hive)

Para cargas masivas diarias donde buscas máxima velocidad y aislamiento total, se utiliza la sentencia `ALTER TABLE ... SWITCH TO`. Esto permite realizar **cargas ETL ultra rápidas** en milisegundos, ya que únicamente intercambia los metadatos de los punteros de almacenamiento en lugar de insertar fila por fila.

Flujo de trabajo en SQL Server:

1. **Tabla de Staging (Área de trabajo):** Ingestas el archivo diario (los 100 registros) en una tabla temporal o de staging sin particionar.
2. **Preparar el corte de fecha:** Aseguras que la partición del día (ej. `2026-10-04`) exista mediante `SPLIT RANGE` en la tabla destino.
3. **Mover los datos (SWITCH):** Haces el `SWITCH` de la tabla de staging directamente a la partición correspondiente de la tabla principal.

Ejemplo de implementación:

SQL

```sql
-- 1. Crear la función de partición por fecha diaria (ejemplo de rangos)
CREATE PARTITION FUNCTION PF_IngestaDiaria (DATE)
AS RANGE RIGHT FOR VALUES
(
    '2026-10-04',
    '2026-10-05',
    '2026-10-06'
);
GO

-- 2. Crear el Esquema de Partición (mapeado al Filegroup deseado)
CREATE PARTITION SCHEME PS_IngestaDiaria
AS PARTITION PF_IngestaDiaria
ALL TO ([PRIMARY]);
GO
```

```
-- 3. Crear la Tabla Principal Particionada
CREATE TABLE dbo.VentasDiarias
(
    VentaID INT NOT NULL,
    Monto DECIMAL(18,2) NOT NULL,
    FechaIngesta DATE NOT NULL,
    CONSTRAINT PK_VentasDiarias PRIMARY KEY CLUSTERED (FechaIngesta, VentaID)
) ON PS_IngestaDiaria(FechaIngesta);
GO

-- 4. Crear la Tabla de Staging (Debe ser IDÉNTICA en estructura y restricciones)
CREATE TABLE dbo.VentasStaging
(
    VentaID INT NOT NULL,
    Monto DECIMAL(18,2) NOT NULL,
    FechaIngesta DATE NOT NULL,
    CONSTRAINT PK_VentasStaging PRIMARY KEY CLUSTERED (FechaIngesta, VentaID),
    CONSTRAINT CK_VentasStaging_Fecha CHECK (FechaIngesta = '2026-10-04')
);
GO

-- 5. Ingestar los datos en Staging y mover a la partición de la Tabla Principal
-- (Carga tu archivo de 100 registros en dbo.VentasStaging primero)

-- Intercambio de metadatos instantáneo hacia la partición 1 (correspondiente a 2
ALTER TABLE dbo.VentasStaging
SWITCH TO dbo.VentasDiarias PARTITION 1;
GO
```

Enfoque 2: Tablas de Particionamiento Manual con Vistas (`Partitioned Views`)

Si la granularidad diaria genera demasiadas particiones físicas (SQL Server soporta hasta 15,000 particiones por tabla, pero gestionar miles de rangos a diario puede requerir mantenimientos frecuentes de `SPLIT`), un patrón muy común en SQL Server cuando se trabaja con archivos ingestados por fecha es el de las **Vistas Particionadas**.

Consiste en tener una tabla por cada período (por ejemplo, por mes o por día si el volumen es gigante) y unirlas mediante una vista lógica:

SQL

```sql
-- Crear tablas por día o por periodo
CREATE TABLE dbo.Ventas_20261004 (
    VentaID INT PRIMARY KEY,
    Monto DECIMAL(18,2),
    FechaIngesta DATE CHECK (FechaIngesta = '2026-10-04')
);

-- Vista unificada para consultar como si fuera una sola tabla en Hive
CREATE VIEW dbo.vw_VentasTotales AS
SELECT * FROM dbo.Ventas_20261004
UNION ALL
SELECT * FROM dbo.Ventas_20261005;
GO
```

> **Ventaja en SQL Server:** *Gracias a las restricciones* `CHECK`*, el optimizador de SQL Server realiza* `Partition Elimination` *de forma automática. Si ejecutas* `SELECT * FROM dbo.vw_VentasTotales WHERE FechaIngesta = '2026-10-04'`*, SQL Server leerá exclusivamente la tabla física de ese día e ignorará las demás.*

Resumen de diferencias clave con Hive

Característica	Apache Hive	SQL Server
Creación de Partición	Dinámica/Automática al insertar datos con un valor de partición nuevo.	Manual/Programada mediante script (`SPLIT RANGE` en la función de partición antes o durante el proceso ETL).
Estructura física	Directorios en HDFS/S3 (`/fecha=20261004/`).	Archivos de datos (`.ndf`) y Filegroups organizados por el motor en disco.

Característica	Apache Hive	SQL Server
Ingesta típica	`INSERT OVERWRITE PARTITION (fecha='20261004')` .	Carga a Staging + `SWITCH PARTITION` o `INSERT` directo en tabla particionada.

¿Te gustaría ver un script de automatización en T-SQL que cree la nueva partición diaria aut

¿Cómo puedo automatizar mediante un procedimiento almacenado en SQL Server la creación de particiones diarias con SPLIT RANGE antes de ingestar los datos?

Para automatizar la creación de particiones diarias en SQL Server mediante un procedimiento almacenado, la lógica debe realizar los siguientes pasos de forma dinámica:

1. **Calcular la fecha objetivo:** Determinar la fecha de la nueva partición (por ejemplo, el día de mañana o una fecha específica ingresada por parámetro).
2. **Comprobar si el rango ya existe:** Consultar las vistas de sistema (`sys.partition_range_values`) para evitar errores por intentar agregar un límite duplicado.
3. **Asignar el Filegroup:** Indicar al `PARTITION SCHEME` cuál será el siguiente Filegroup a utilizar mediante `NEXT USED` .
4. **Dividir el rango:** Ejecutar el `SPLIT RANGE` en la `PARTITION FUNCTION` con SQL Dinámico (`EXEC sp_executesql`).

Procedimiento Almacenado de Automatización

Este script crea un procedimiento almacenado robusto que se encarga de crear el rango para una fecha dada (o por defecto para el día siguiente si no se pasa parámetro):

SQL

```sql
CREATE OR ALTER PROCEDURE dbo.usp_CrearParticionDiaria
    @FechaParticion DATE = NULL -- Si es NULL, genera la del día siguiente
AS
```

```
BEGIN
    SET NOCOUNT ON;

    -- 1. Si no se pasa fecha, tomar por defecto la fecha de mañana
    IF @FechaParticion IS NULL
        SET @FechaParticion = DATEADD(DAY, 1, CAST(GETDATE() AS DATE));

    DECLARE @FechaStr VARCHAR(10) = CONVERT(VARCHAR(10), @FechaParticion, 120);
    DECLARE @NombreFunción NVARCHAR(128) = N'PF_IngestaDiaria';
    DECLARE @NombreEsquema NVARCHAR(128) = N'PS_IngestaDiaria';
    DECLARE @NombreFilegroup NVARCHAR(128) = N'PRIMARY'; -- O el Filegroup destin
    DECLARE @SQL NVARCHAR(MAX);

    -- 2. Verificar si el límite de fecha ya existe en la Función de Partición
    IF EXISTS (
        SELECT 1
        FROM sys.partition_range_values rv
        INNER JOIN sys.partition_functions pf ON rv.function_id = pf.function_id
        WHERE pf.name = @NombreFunción
          AND CAST(rv.value AS DATE) = @FechaParticion
    )
    BEGIN
        PRINT 'La partición para la fecha ' + @FechaStr + ' ya existe. No se real
        RETURN;
    END

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 3. Marcar el Filegroup como NEXT USED en el Partition Scheme
        SET @SQL = N'ALTER PARTITION SCHEME ' + QUOTENAME(@NombreEsquema) +
                   N' NEXT USED ' + QUOTENAME(@NombreFilegroup) + N';';
        EXEC sp_executesql @SQL;

        -- 4. Ejecutar el SPLIT RANGE en la Partition Function
        SET @SQL = N'ALTER PARTITION FUNCTION ' + QUOTENAME(@NombreFunción) +
                   N'() SPLIT RANGE (''' + @FechaStr + N''');';
        EXEC sp_executesql @SQL;

        COMMIT TRANSACTION;
        PRINT 'Partición para la fecha ' + @FechaStr + ' creada exitosamente.';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
```

```
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END;
GO
```

Ejemplo de Ejecución y Automatización

Para ejecutar el procedimiento almacenado manualmente o programarlo en un job de **SQL Server Agent**:

SQL

```
-- Crear la partición para el día de mañana automáticamente
EXEC dbo.usp_CrearParticionDiaria;

-- Crear una partición para una fecha específica
EXEC dbo.usp_CrearParticionDiaria @FechaParticion = '2026-10-05';
```

Estrategia de Programación recomendada

Para garantizar que tus ingestas nunca fallen por falta de espacio de particionamiento:

1. **SQL Server Agent Job:** Programa la ejecución de este procedimiento para que se ejecute diariamente (por ejemplo, a las 23:00 hrs) o semanalmente creando por adelantado los rangos de los próximos 7 días con un bucle.
2. **Ejecución previa en el Job de ETL:** Agrega este `EXEC` como el **Paso 1** en tu flujo o pipeline de carga (SSIS, Azure Data Factory o scripts de Python/PowerShell) antes de ejecutar la ingesta a la tabla de staging y el `SWITCH PARTITION`.

¿Quieres ver cómo configurar un bucle dentro del procedimiento para crear por adelantado las particiones de todo un mes?

↪ Sí

quiesiera saber mas bien la configuracion del procedimiento pero para que cree la particion al momento de la ingesta y no al futuro. Ademas quiero saber una vez que se cree la particion , como seria el query de insercion de los

Para manejar este flujo **"Just-in-Time"** (crear la partición en el momento exacto de la ingesta), el procedimiento almacenado debe verificar la fecha del archivo a cargar, garantizar que la partición exista, y luego ejecutar la inserción o el reemplazo de datos.

1. Procedimiento Almacenado de Ingesta Just-in-Time

Este procedimiento recibe la fecha del archivo que estás procesando. Si la partición no existe la crea en ese instante; luego, gestiona la carga o reemplazo de datos.

SQL

```sql
CREATE OR ALTER PROCEDURE dbo.usp_IngresarDatosDiarios
    @FechaArchivo DATE,
    @ReemplazarExistentes BIT = 0 -- 0: Solo insertar | 1: Sobrescribir partición
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @FechaStr VARCHAR(10) = CONVERT(VARCHAR(10), @FechaArchivo, 120);
    DECLARE @FechaSiguienteStr VARCHAR(10) = CONVERT(VARCHAR(10), DATEADD(DAY, 1,
    DECLARE @NombreFunción NVARCHAR(128) = N'PF_IngestaDiaria';
    DECLARE @NombreEsquema NVARCHAR(128) = N'PS_IngestaDiaria';
    DECLARE @NombreFilegroup NVARCHAR(128) = N'PRIMARY';
    DECLARE @SQL NVARCHAR(MAX);

    BEGIN TRY
        BEGIN TRANSACTION;

        ----------------------------------------------------------------
        -- PASO 1: Garantizar la existencia de la partición (SPLIT)
        ----------------------------------------------------------------
        -- Se requieren dos cortes si no existen: el límite inferior (fecha) y el
        IF NOT EXISTS (
            SELECT 1 FROM sys.partition_range_values rv
```

```
    INNER JOIN sys.partition_functions pf ON rv.function_id = pf.function
    WHERE pf.name = @NombreFunción AND CAST(rv.value AS DATE) = @FechaArc
)
BEGIN
    -- Marcar Filegroup y hacer SPLIT para el inicio del día
    SET @SQL = N'ALTER PARTITION SCHEME ' + QUOTENAME(@NombreEsquema) + N
    EXEC sp_executesql @SQL;

    SET @SQL = N'ALTER PARTITION FUNCTION ' + QUOTENAME(@NombreFunción) +
    EXEC sp_executesql @SQL;
END

-- Corte para el día siguiente (asegura el límite superior del rango)
IF NOT EXISTS (
    SELECT 1 FROM sys.partition_range_values rv
    INNER JOIN sys.partition_functions pf ON rv.function_id = pf.function
    WHERE pf.name = @NombreFunción AND CAST(rv.value AS DATE) = DATEADD(D
)
BEGIN
    SET @SQL = N'ALTER PARTITION SCHEME ' + QUOTENAME(@NombreEsquema) + N
    EXEC sp_executesql @SQL;

    SET @SQL = N'ALTER PARTITION FUNCTION ' + QUOTENAME(@NombreFunción) +
    EXEC sp_executesql @SQL;
END

-----------------------------------------------------------------------
-- PASO 2: Manejo de reemplazo de datos (Reescribir partición)
-----------------------------------------------------------------------
IF @ReemplazarExistentes = 1
BEGIN
    -- Se eliminan únicamente los registros pertenecientes a esa fecha
    DELETE FROM dbo.VentasDiarias
    WHERE FechaIngesta = @FechaArchivo;
END

-----------------------------------------------------------------------
-- PASO 3: Inserción de datos desde la tabla de Staging
-----------------------------------------------------------------------
INSERT INTO dbo.VentasDiarias (VentaID, Monto, FechaIngesta)
SELECT VentaID, Monto, FechaIngesta
FROM dbo.VentasStaging
WHERE FechaIngesta = @FechaArchivo;

-- Limpiar el área de staging tras la carga exitosa
TRUNCATE TABLE dbo.VentasStaging;
```

```
        COMMIT TRANSACTION;
        PRINT 'Ingesta completada exitosamente para la fecha: ' + @FechaStr;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END;
GO
```

2. ¿Cómo sería la consulta de inserción (Carga Estándar)?

Primero vacías tu tabla `Staging`, insertas las 100 filas del archivo del día (vía `BULK INSERT`, SSIS, Python o `INSERT INTO`), y ejecutas el procedimiento especificando que **no** deseas reemplazar nada (`@ReemplazarExistentes = 0`):

SQL

```
-- 1. Cargas tus 100 registros a la tabla temporal de Staging
INSERT INTO dbo.VentasStaging (VentaID, Monto, FechaIngesta) VALUES
(1001, 250.00, '2026-10-04'),
(1002, 180.50, '2026-10-04');

-- 2. Ejecutas la ingesta Just-In-Time
EXEC dbo.usp_IngresarDatosDiarios
    @FechaArchivo = '2026-10-04',
    @ReemplazarExistentes = 0;
```

3. ¿Qué pasa si necesitas REEMPLAZAR los datos de esa partición?

Si necesitas reingestar un archivo corregido del mismo día (equivalente al `INSERT OVERWRITE` de Hive), existen **dos estrategias** en SQL Server según el volumen de datos:

Opción A: Vía `DELETE` dirigido (Ideal para volúmenes pequeños o medianos)

Es el método utilizado dentro del procedimiento almacenado arriba cuando pasas `@ReemplazarExistentes = 1`:

SQL

```sql
-- Ejecuta la ingesta borrando previamente los datos de esa fecha
EXEC dbo.usp_IngresarDatosDiarios
    @FechaArchivo = '2026-10-04',
    @ReemplazarExistentes = 1;
```

> **¿Qué ocurre internamente?**
> *Gracias a las restricciones y a la columna de partición `FechaIngesta`, el optimizador de SQL Server realiza* **Partition Elimination.** *El comando `DELETE` se dirige únicamente a las páginas de memoria/disco que contienen la partición del `'2026-10-04'`, dejando intacto el resto de las particiones de la tabla.*

Opción B: Vía `TRUNCATE TARGET_PARTITION` (Ideal para volúmenes masivos)

Si la partición contiene millones de filas y un `DELETE` genera demasiados registros en el log de transacciones, a partir de SQL Server 2016 se puede vaciar únicamente una partición específica mediante metadatos:

SQL

```sql
-- 1. Obtener el número de partición exacto para la fecha
DECLARE @PartitionNumber INT;

SELECT @PartitionNumber = $PARTITION.PF_IngestaDiaria('2026-10-04');

-- 2. Vaciar EXCLUSIVAMENTE esa partición en milisegundos sin generar log extenso
TRUNCATE TABLE dbo.VentasDiarias
WITH (PARTITIONS (@PartitionNumber));

-- 3. Insertar los nuevos datos corregidos
INSERT INTO dbo.VentasDiarias (VentaID, Monto, FechaIngesta)
```

```
SELECT VentaID, Monto, FechaIngesta
FROM dbo.VentasStaging;
```

¿Quieres ver cómo adaptar esta lógica usando SWITCH PARTITION para reemplazos instantáneos de gran volumen?

↪ Sí

que pasa, si necesito reemplazar los valores de una particion que ya existia

Cuando necesitas reemplazar los datos de una partición que **ya existe** en SQL Server (el equivalente a un `INSERT OVERWRITE` en Hive), tienes **tres alternativas principales**, dependiendo del volumen de datos que estés manejando y de qué tan crítica sea la velocidad.

Opción 1: `TRUNCATE TABLE ... WITH (PARTITIONS (...))`

(Recomendada: Rápida, limpia y moderna)

A partir de **SQL Server 2016**, puedes vaciar únicamente una partición específica mediante metadatos (sin generar bloqueos extensos ni llenar el Log de Transacciones con cada fila borrada).

¿Cómo funciona?

1. Usas la función `$PARTITION` para calcular automáticamente el número de partición de la fecha objetivo.
2. Vacias exclusivamente esa partición con `TRUNCATE` .
3. Insertas los nuevos datos desde la tabla de staging.

SQL

```
DECLARE @FechaObjetivo DATE = '2026-10-04';
DECLARE @NumParticion INT;

-- 1. Identificar el número de partición asociado a la fecha
SELECT @NumParticion = $PARTITION.PF_IngestaDiaria(@FechaObjetivo);

BEGIN TRANSACTION;

    -- 2. Vaciar ÚNICAMENTE esa partición (operación instantánea a nivel de metad
    DECLARE @SQL NVARCHAR(MAX) = N'TRUNCATE TABLE dbo.VentasDiarias WITH (PARTITI
    EXEC sp_executesql @SQL;

    -- 3. Insertar los datos reemplazantes desde Staging
    INSERT INTO dbo.VentasDiarias (VentaID, Monto, FechaIngesta)
    SELECT VentaID, Monto, FechaIngesta
    FROM dbo.VentasStaging
    WHERE FechaIngesta = @FechaObjetivo;

    -- 4. Limpiar la tabla de Staging
    TRUNCATE TABLE dbo.VentasStaging;

COMMIT TRANSACTION;
```

Opción 2: `SWITCH PARTITION` con Tabla Vacía

(Para alta disponibilidad: Cero tiempo de inactividad / Zero Downtime)

Si la partición existente contiene millones de filas y **no quieres que los usuarios vean la partición vacía ni por un milisegundo** mientras se realiza la nueva carga, usas la técnica de *Switching*.

Pasos del proceso:

1. Creas una tabla auxiliar temporal o de intercambio (`dbo.VentasBasura`) con la misma estructura e índices.
2. Intercambias la partición existente hacia la tabla de basura (la partición de la tabla principal queda instantáneamente vacía).
3. Haces el `SWITCH` de la tabla de Staging (donde ya cargaste los nuevos datos) directamente hacia esa partición.

SQL

```sql
DECLARE @FechaObjetivo DATE = '2026-10-04';
DECLARE @NumParticion INT = $PARTITION.PF_IngestaDiaria(@FechaObjetivo);

BEGIN TRANSACTION;

    -- 1. Sacar los datos viejos fuera de la tabla principal a una tabla de desca
    ALTER TABLE dbo.VentasDiarias
    SWITCH PARTITION @NumParticion TO dbo.VentasBasura;

    -- 2. Vaciar la tabla de descarte
    TRUNCATE TABLE dbo.VentasBasura;

    -- 3. Mover los nuevos datos desde Staging a la partición de la tabla princip
    ALTER TABLE dbo.VentasStaging
    SWITCH TO dbo.VentasDiarias PARTITION @NumParticion;

COMMIT TRANSACTION;
```

Opción 3: `DELETE` filtrado por fecha

(Para volúmenes pequeños a medianos)

Si la carga es pequeña (como en el ejemplo de los 100 registros), un `DELETE` estándar filtrado por la fecha funciona perfectamente.

SQL

```sql
BEGIN TRANSACTION;

    -- SQL Server hace "Partition Elimination" y solo toca las páginas de esa fec
    DELETE FROM dbo.VentasDiarias
    WHERE FechaIngesta = '2026-10-04';

    INSERT INTO dbo.VentasDiarias (VentaID, Monto, FechaIngesta)
    SELECT VentaID, Monto, FechaIngesta
    FROM dbo.VentasStaging;
```

```
COMMIT TRANSACTION;
```

Resumen de Comparación

Método	Velocidad	Uso de Log de Transacciones	Disponibilidad d Lectura
`TRUNCATE PARTITION`	Casi instantáneo (ms)	Mínimo (Múltiples de página)	Breve ventana v durante el `INSI`
`SWITCH PARTITION`	Instantáneo (ms)	Mínimo (Solo metadatos)	**Zero Downtime** atómico de dat viejos a nuevos)
`DELETE`	Depende del volumen	Alto (Registra fila por fila)	Bloqueos temp son muchos dat

¿Quieres que integramos el método TRUNCATE PARTITION dentro del procedimiento almacenado de ingesta diaria?

↪ Sí

muestra

ese archivo continene la informacion del la tabla que quiero particionar. Actua como un experto en sql server. El problema a modelar es crear una base de datos en sql y dentro de ella una tabla donde pueda cargar este

Mostrar código <>

Para emular el comportamiento de **Apache Hive** en **SQL Server** (donde las ingestas se realizan agregando, actualizando o reemplazando particiones completas por fecha en formato `YYYYMMDD` o `DATE`), implementaremos una arquitectura basada en:

1. **Tabla de Staging (Área de Ingesta):** Permite recibir el archivo diario sin afectar la tabla analítica.
2. **Particionamiento por Rangos Dinámicos (`INT` en formato `YYYYMMDD` o `DATE`):** Garantiza que los datos de cada día estén aislados físicamente.
3. **Procedimiento Almacenado de Carga "Just-In-Time":** Verifica si la partición de la fecha enviada (ej. `20261001`) existe. Si no existe, ejecuta el `SPLIT RANGE` automáticamente. Si ya existe y se requiere sobreescribir (estilo `INSERT OVERWRITE`), ejecuta `TRUNCATE TABLE ... WITH (PARTITIONS)` sobre esa fecha específica y recarga los datos en milisegundos.

Paso 1: Crear la Base de Datos y la Estructura de Particionamiento

Definimos una **Partition Function** de tipo `INT` para manejar fechas enteras en formato `YYYYMMDD` (ej. `20261001`), que facilita la integración directa con los sistemas de archivos de Hive.

SQL

```sql
USE master;
GO

IF EXISTS (SELECT name FROM sys.databases WHERE name = N'DB_Creditos')
    DROP DATABASE DB_Creditos;
GO

CREATE DATABASE DB_Creditos;
GO

USE DB_Creditos;
GO
```

```
-- 1. Crear la Función de Partición basada en fecha numérica (YYYYMMDD)
-- Se inicializa con valores de frontera base
CREATE PARTITION FUNCTION PF_Creditos_Fecha (INT)
AS RANGE RIGHT FOR VALUES (20261001, 20261002);
GO

-- 2. Crear el Esquema de Partición asociado al Filegroup primario
CREATE PARTITION SCHEME PS_Creditos_Fecha
AS PARTITION PF_Creditos_Fecha
ALL TO ([PRIMARY]);
GO
```

Paso 2: Crear la Tabla Particionada y la Tabla Staging

Creamos el modelo de datos basado en las columnas de la muestra del reporte de créditos recibida, incorporando la columna `FechaReporte INT` como clave de partición.

SQL

```
-- 1. Tabla Principal Particionada (Destino Final)
CREATE TABLE dbo.FactCreditos
(
    FechaReporte INT NOT NULL, -- Campo de Partición (Ej: 20261001)
    TipoPersonaRIF VARCHAR(5),
    IdentificacionTipoPersonaRIF BIGINT,
    NombreRazonSocial VARCHAR(255),
    ActividadCliente VARCHAR(50),
    TipoCredito INT,
    FechaAprobacionCredito INT,
    FechaVencimientoCredito INT,
    NumeroCredito BIGINT NOT NULL,
    CodigoCuentaContable BIGINT,
    MontoOriginalBolivares DECIMAL(18,2),
    MontoOriginalPetro VARCHAR(50),
    DestinoRecursos VARCHAR(100),
    Rubro VARCHAR(100),
    EntidadFederal VARCHAR(100),
    NumeroDesembolsos INT,
    MontoLiquidadoBolivares DECIMAL(18,2),
    MontoLiquidadoPetro VARCHAR(50),
    UnidadValorCreditoCantidad VARCHAR(50),
```

```
    FechaLiquidacion INT,
    SituacionCredito VARCHAR(50),
    FechaSituacionCredito INT,
    PorcentajeAmortizacion DECIMAL(5,2),
    MontoAmortizacionBolivares DECIMAL(18,2),
    MontoAmortizacionPetro VARCHAR(50),
    SaldoCreditoBolivares DECIMAL(18,2),
    SaldoCreditoPetro VARCHAR(50),

    -- La clave primaria DEBE incluir el campo de partición
    CONSTRAINT PK_FactCreditos PRIMARY KEY CLUSTERED (FechaReporte, NumeroCredito
) ON PS_Creditos_Fecha(FechaReporte);
GO

-- 2. Tabla Staging (Estructura idéntica, no particionada, para carga temporal)
CREATE TABLE dbo.Staging_FactCreditos
(
    FechaReporte INT NOT NULL,
    TipoPersonaRIF VARCHAR(5),
    IdentificacionTipoPersonaRIF BIGINT,
    NombreRazonSocial VARCHAR(255),
    ActividadCliente VARCHAR(50),
    TipoCredito INT,
    FechaAprobacionCredito INT,
    FechaVencimientoCredito INT,
    NumeroCredito BIGINT NOT NULL,
    CodigoCuentaContable BIGINT,
    MontoOriginalBolivares DECIMAL(18,2),
    MontoOriginalPetro VARCHAR(50),
    DestinoRecursos VARCHAR(100),
    Rubro VARCHAR(100),
    EntidadFederal VARCHAR(100),
    NumeroDesembolsos INT,
    MontoLiquidadoBolivares DECIMAL(18,2),
    MontoLiquidadoPetro VARCHAR(50),
    UnidadValorCreditoCantidad VARCHAR(50),
    FechaLiquidacion INT,
    SituacionCredito VARCHAR(50),
    FechaSituacionCredito INT,
    PorcentajeAmortizacion DECIMAL(5,2),
    MontoAmortizacionBolivares DECIMAL(18,2),
    MontoAmortizacionPetro VARCHAR(50),
    SaldoCreditoBolivares DECIMAL(18,2),
    SaldoCreditoPetro VARCHAR(50)
```

```
);
GO
```

Paso 3: Crear el Procedimiento Almacenado de Ingesta y Reemplazo de Particiones

Este procedimiento automatiza completamente el flujo estilo Hive:

Comprueba si la fecha enviada existe como partición. Si no existe, realiza un `SPLIT RANGE` para crearla.

Si se activa `@ModoSobrescribir = 1` (INSERT OVERWRITE), ejecuta un `TRUNCATE` exclusivo sobre esa partición mediante metadatos para vaciarla al instante sin afectar otras fechas.

Copia los datos desde la tabla `Staging_FactCreditos` a `FactCreditos`.

SQL

```sql
CREATE OR ALTER PROCEDURE dbo.usp_IngresarCreditosDiarios
    @FechaReporte INT,           -- Fecha en formato YYYYMMDD (Ej: 20261001)
    @ModoSobrescribir BIT = 1    -- 1: Reemplazar partición (Hive OVERWRITE) | 0:
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NombreFuncion NVARCHAR(128) = N'PF_Creditos_Fecha';
    DECLARE @NombreEsquema NVARCHAR(128) = N'PS_Creditos_Fecha';
    DECLARE @NombreFilegroup NVARCHAR(128) = N'PRIMARY';
    DECLARE @NumParticion INT;
    DECLARE @SQL NVARCHAR(MAX);

    BEGIN TRY
        BEGIN TRANSACTION;

        ----------------------------------------------------------------
        -- 1. Garantizar la existencia de la partición (SPLIT dinámico)
        ----------------------------------------------------------------
        IF NOT EXISTS (
            SELECT 1 FROM sys.partition_range_values rv
            INNER JOIN sys.partition_functions pf ON rv.function_id = pf.function
```

```
            WHERE pf.name = @NombreFuncion AND CAST(rv.value AS INT) = @FechaRepo
        )
        BEGIN
            -- Indicar el siguiente Filegroup a utilizar
            SET @SQL = N'ALTER PARTITION SCHEME ' + QUOTENAME(@NombreEsquema) + N
            EXEC sp_executesql @SQL;

            -- Crear la nueva frontera de la partición
            SET @SQL = N'ALTER PARTITION FUNCTION ' + QUOTENAME(@NombreFuncion) +
            EXEC sp_executesql @SQL;
        END

        ----------------------------------------------------------------------
        -- 2. Reemplazo/Sobrescritura de la partición si ya contenía datos
        ----------------------------------------------------------------------
        IF @ModoSobrescribir = 1
        BEGIN
            -- Obtener el número físico de la partición asignada a la fecha
            SELECT @NumParticion = $PARTITION.PF_Creditos_Fecha(@FechaReporte);

            -- Truncar únicamente los datos de esa partición específica (operació
            SET @SQL = N'TRUNCATE TABLE dbo.FactCreditos WITH (PARTITIONS (' + CA
            EXEC sp_executesql @SQL;
        END

        ----------------------------------------------------------------------
        -- 3. Inserción de los datos cargados desde la Staging
        ----------------------------------------------------------------------
        INSERT INTO dbo.FactCreditos
        SELECT * FROM dbo.Staging_FactCreditos
        WHERE FechaReporte = @FechaReporte;

        -- Limpieza de la tabla de trabajo
        TRUNCATE TABLE dbo.Staging_FactCreditos;

        COMMIT TRANSACTION;
        PRINT 'Ingesta completada exitosamente para la partición: ' + CAST(@Fecha
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
```

```
END;
GO
```

Paso 4: Ejemplo de Uso Práctico (Carga y Reemplazo)

1. Ingesta Inicial del archivo con fecha `20261001`:

SQL

```
-- Cargas el archivo de Excel/CSV a la tabla Staging (vía SSIS, BCP, Python o INS
INSERT INTO dbo.Staging_FactCreditos (FechaReporte, TipoPersonaRIF, Identificacio
VALUES (20261001, 'J', 15822, 'THE FACTORY HKA, C.A', 17607, 2200000.00);

-- Ejecutas el proceso de ingesta
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

2. Reemplazo completo de la partición `20261001` (Si el archivo fue corregido):

SQL

```
-- Cargas la nueva versión del archivo a la Staging
INSERT INTO dbo.Staging_FactCreditos (FechaReporte, TipoPersonaRIF, Identificacio
VALUES (20261001, 'J', 15822, 'THE FACTORY HKA, C.A CORREGIDO', 17607, 2500000.00

-- Se vuelve a ejecutar el procedimiento: vaciará la partición 20261001 e inserta
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

3. Eliminar la partición `20261001` directamente si es necesario:

Si en algún momento solo deseas purgar o vaciar la partición sin reingestar datos:

```sql
DECLARE @NumParticion INT = $PARTITION.PF_Creditos_Fecha(20261001);

-- Elimina los registros de esa fecha en milisegundos sin afectar el resto de la
TRUNCATE TABLE dbo.FactCreditos WITH (PARTITIONS (@NumParticion));
```

↪ USE master; GO

porque usas master como la base de datos , si quiero crear una base datos diferente

Mostrar código <>

Se utiliza `USE master;` en ese primer paso por una razón de arquitectura y seguridad dentro de SQL Server:

1. La base de datos `master` es el catálogo del sistema

En SQL Server, la base de datos `master` almacena toda la información a nivel de servidor (listado de bases de datos, inicios de sesión, configuraciones). Para poder ejecutar la sentencia `CREATE DATABASE` (o `DROP DATABASE`), **tu sesión no puede estar ubicada dentro de la misma base de datos que intentas crear o eliminar**.

2. Evita errores de "Base de datos en uso" (`Database in use`)

Si intentas eliminar o recrear una base de datos estando posicionado dentro de ella, SQL Server devolverá un error impidiendo la operación porque la base de datos estará bloqueada por tu propia conexión.

El flujo estándar en scripts DDL es:

1. Cambiar al contexto global con `USE master;` .
2. Crear la nueva base de datos (ej. `CREATE DATABASE MiOtraBaseDeDatos;`).

3. Cambiar inmediatamente al contexto de la nueva base de datos recién creada mediante `USE MiOtraBaseDeDatos;` para crear allí sus objetos (filegroups, funciones de partición, tablas y procedimientos almacenados).

Ejemplo ajustado con el nombre de base de datos que tú elijas

Puedes cambiar el nombre de la base de datos por cualquier otro (por ejemplo `DB_Finanzas` o `DB_Ingestas`). `master` solo se usa de forma temporal durante la creación inicial:

SQL

```sql
-- 1. Cambias a master solo para administrar la creación a nivel de servidor
USE master;
GO

-- 2. Creas tu base de datos con el nombre personalizado que prefieras
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'DB_Finanzas')
    DROP DATABASE DB_Finanzas;
GO

CREATE DATABASE DB_Finanzas;
GO

-- 3. Cambias a tu nueva base de datos. TODO lo que ejecutes a partir de aquí
-- (Filegroups, Partition Functions, Tablas) se creará dentro de DB_Finanzas
USE DB_Finanzas;
GO

-- A partir de aquí continúas con la creación de las particiones y tablas
CREATE PARTITION FUNCTION PF_Creditos_Fecha (INT)
AS RANGE RIGHT FOR VALUES (20261001, 20261002);
GO
```

porque no veo la tabla creada

Mostrar código <>

Si ejecutaste los scripts y no ves la tabla creada en tu entorno de SQL Server Management Studio (SSMS) o Azure Data Studio, esto suele deberse a uno de los siguientes **4 motivos principales**:

1. El contexto de la sesión (No cambiar a la base de datos correcta)

Cuando ejecutas un script que contiene `USE master;` para crear la base de datos, si no ejecutas la línea `USE TuBaseDeDatos;` inmediatamente después, la tabla se habrá creado en el contexto equivocado (frecuentemente dentro de la base de datos `master` en lugar de tu nueva base de datos).

Solución:
Asegúrate de seleccionar tu base de datos en el menú desplegable superior de SSMS o ejecuta:

SQL

```sql
USE DB_Finanzas; -- Cambia DB_Finanzas por el nombre de tu base de datos
GO

-- Verifica si la tabla existe en esta base de datos
SELECT * FROM sys.tables WHERE name = 'FactCreditos';
```

2. No se ha actualizado el Explorador de Objetos (Falta de Refresh)

En SQL Server Management Studio (SSMS), el árbol de objetos a la izquierda **no se actualiza automáticamente** tras ejecutar un script DDL (`CREATE TABLE`).

Solución:

1. Ve al **Object Explorer** (Explorador de Objetos) a la izquierda.
2. Despliega `Databases` -> `TuBaseDeDatos` -> `Tables` .
3. Haz clic derecho sobre la carpeta **Tables** y selecciona **Refresh** (o presiona la tecla **F5** seleccionando la carpeta).

3. Error en la creación de los Filegroups o Rutas de Archivos

Si la creación de la función o esquema de partición falló por un error de sintaxis o de permisos de disco, SQL Server interrumpe el script y **no llega a ejecutar la sentencia** `CREATE TABLE`.

Solución:
Revisa la pestaña de **Messages** (Mensajes) en la parte inferior de la ventana de consulta para verificar si hubo un mensaje de error en rojo. Si ocurrió un error en un bloque previo, la tabla no se habrá llegado a instanciar.

4. Búsqueda mediante la consulta de sistema

Para salir de dudas y saber en qué base de datos física del servidor quedó guardada tu tabla, ejecuta esta consulta global:

SQL

```
-- Buscar la tabla en todas las bases de datos de tu instancia
EXEC sp_MSforeachdb '
    USE [?];
    IF EXISTS (SELECT 1 FROM sys.tables WHERE name = ''FactCreditos'')
        PRINT ''La tabla se encuentra en la base de datos: '' + DB_NAME()
';
```

Si la consulta devuelve el nombre de una base de datos distinta a la que esperabas, simplemente conéctate a esa base de datos o vuelve a ejecutar el script asegurándote de posicionarte en la base de datos correcta desde la primera línea.

ya estoy viendo la base de datos creada, la tabla y la tabla Staging_FactCreditos, cuando consulto la tabla esta vacia a pesar de que ya ejecute este script que me indicastes INSERT INTO dbo.Staging_FactCreditos (FechaRep

Mostrar código <>

Esto que observas ocurre exactamente por cómo está diseñado el flujo ETL y por la diferencia entre la tabla temporal de trabajo (`Staging_FactCreditos`) y la tabla particionada final (`FactCreditos`).

Aquí te explico detalladamente los **dos fenómenos** que estás experimentando y por qué el comportamiento es el esperado:

1. ¿Por qué la tabla `FactCreditos` sigue vacía después del `INSERT` ?

El script `INSERT INTO dbo.Staging_FactCreditos ...` inserta los registros únicamente en la **tabla Staging (la zona de descarga temporal), NO** en la tabla final `FactCreditos` .

Para que los datos pasen de la tabla `Staging` a la tabla particionada `FactCreditos` , **debes ejecutar el procedimiento almacenado** `usp_IngresarCreditosDiarios` :

El flujo completo es de dos pasos:

SQL

```sql
-- PASO A: Insertar los datos en la tabla Staging (Zona temporal)
INSERT INTO dbo.Staging_FactCreditos (FechaReporte, TipoPersonaRIF, Identificacio
VALUES (20261001, 'J', 15822, 'THE FACTORY HKA, C.A CORREGIDO', 17607, 2500000.00

-- PASO B: Ejecutar la ingesta para que los mueva a FactCreditos y aplique la par
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

> **Nota clave:** *Al final del procedimiento almacenado hay una sentencia* `TRUNCATE TABLE dbo.Staging_FactCreditos;` *. Por lo tanto, después de mover los datos a* `FactCreditos` *, la tabla* `Staging` *se vacía automáticamente para quedar lista para el archivo del día siguiente.*

2. ¿Por qué el `TRUNCATE TABLE WITH (PARTITIONS)` no borró nada?

En tu consulta ejecutaste:

SQL

```sql
TRUNCATE TABLE dbo.FactCreditos WITH (PARTITIONS (@NumParticion));
```

Ocurrieron dos cosas aquí:

1. **Intentaste borrar sobre** `FactCreditos` **cuando la tabla todavía estaba vacía:** Como nunca ejecutaste el `EXEC dbo.usp_IngresarCreditosDiarios`, la tabla `FactCreditos` aún no tenía registros guardados en esa partición.
2. **Confusión de tablas:** Mencionas que querías borrar la tabla `Staging_FactCreditos`. El comando `TRUNCATE ... WITH (PARTITIONS)` **solo funciona en tablas particionadas** (como `FactCreditos`). La tabla `Staging_FactCreditos` **no es particionada**, es una tabla normal de paso.

Prueba completa guiada paso a paso

Para que veas el flujo funcionando de principio a fin, ejecuta los siguientes comandos en orden:

Paso 1: Consultar el estado inicial (Ambas tablas deben estar vacías)

SQL

```sql
SELECT * FROM dbo.Staging_FactCreditos; -- Debería estar vacía
SELECT * FROM dbo.FactCreditos;         -- Debería estar vacía
```

Paso 2: Cargar el dato a Staging

SQL

```sql
INSERT INTO dbo.Staging_FactCreditos (FechaReporte, TipoPersonaRIF, Identificacio
VALUES (20261001, 'J', 15822, 'THE FACTORY HKA, C.A CORREGIDO', 17607, 2500000.00
```

```sql
-- Comprobar que el dato está en Staging
SELECT * FROM dbo.Staging_FactCreditos;
```

Paso 3: Ejecutar la Ingesta (El motor procesa la partición)

SQL

```sql
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

Paso 4: Comprobar el resultado final

SQL

```sql
SELECT * FROM dbo.Staging_FactCreditos; -- Ahora está VACÍA (se limpió automática
SELECT * FROM dbo.FactCreditos;         -- Ahora TIENE EL REGISTRO asignado a la
```

Paso 5: Probar la eliminación exclusiva de esa partición

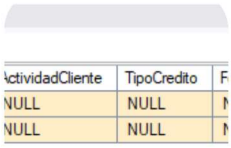
Ahora que `FactCreditos` sí contiene datos en la partición `20261001`, ejecuta la purga por partición:

SQL

```sql
DECLARE @NumParticion INT = $PARTITION.PF_Creditos_Fecha(20261001);

-- Vacía únicamente la partición 20261001 de la tabla final
TRUNCATE TABLE dbo.FactCreditos WITH (PARTITIONS (@NumParticion));
```

```
-- Verificar que se eliminó de FactCreditos
SELECT * FROM dbo.FactCreditos;
```



ctividadCliente	TipoCredito	F
NULL	NULL	N
NULL	NULL	N

actualmente mi tabla Staging_FactCreditos tiene 2 registros como se ve en la tabla al ejecutar EXEC dbo.usp_IngresarCreditosDiarios @FechaReporte = 20261001,

El error de clave primaria (`Violation of PRIMARY KEY constraint`) ocurre porque tu tabla destino `FactCreditos` tiene una restricción de clave primaria compuesta: image_d8f3ff.png

SQL

```
CONSTRAINT PK_FactCreditos PRIMARY KEY CLUSTERED (FechaReporte, NumeroCredito)
```

En la imagen de tu tabla `Staging_FactCreditos` se observa que tienes **2 filas exactamente idénticas:** image_d8f3ff.png

- Fila 1: `FechaReporte = 20261001` y `NumeroCredito = 17607` image_d8f3ff.png
- Fila 2: `FechaReporte = 20261001` y `NumeroCredito = 17607` image_d8f3ff.png

Al ejecutar el `INSERT INTO dbo.FactCreditos SELECT * FROM dbo.Staging_FactCreditos` , SQL Server intenta insertar ambas filas al mismo tiempo, generando un duplicado de la combinación (`20261001` , `17607`) y violando la regla de unicidad de la Primary Key. image_d8f3ff.png

Soluciones

Opción A: Limpiar la tabla de Staging cargando únicamente registros únicos (Recomendado)

Para solucionar el problema actual de tu Staging, elimina los duplicados y deja una sola fila antes de ejecutar el procedimiento:

SQL

```sql
-- 1. Eliminar duplicados en Staging dejando solo 1 registro único
WITH CTE_Duplicados AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY FechaReporte, NumeroCredito
               ORDER BY (SELECT NULL)
           ) AS RowNum
    FROM dbo.Staging_FactCreditos
)
DELETE FROM CTE_Duplicados WHERE RowNum > 1;

-- 2. Ejecutar la ingesta nuevamente
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

Opción B: Proteger el Procedimiento Almacenado contra duplicados en el archivo origen

Si en el día a día los archivos de ingesta (Excel/CSV) pueden llegar accidentalmente con filas repetidas, puedes ajustar el procedimiento almacenado para que al hacer el `INSERT` aplique un `DISTINCT` o `ROW_NUMBER()`, garantizando que nunca falle por duplicados en Staging.

Modifica el **Paso 3** dentro del procedimiento almacenado `dbo.usp_IngresarCreditosDiarios`:

SQL

```
ALTER PROCEDURE dbo.usp_IngresarCreditosDiarios
    @FechaReporte INT,
    @ModoSobrescribir BIT = 1
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NombreFuncion NVARCHAR(128) = N'PF_Creditos_Fecha';
    DECLARE @NombreEsquema NVARCHAR(128) = N'PS_Creditos_Fecha';
    DECLARE @NombreFilegroup NVARCHAR(128) = N'PRIMARY';
    DECLARE @NumParticion INT;
    DECLARE @SQL NVARCHAR(MAX);

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1. Garantizar la partición (SPLIT)
        IF NOT EXISTS (
            SELECT 1 FROM sys.partition_range_values rv
            INNER JOIN sys.partition_functions pf ON rv.function_id = pf.function
            WHERE pf.name = @NombreFuncion AND CAST(rv.value AS INT) = @FechaRepo
        )
        BEGIN
            SET @SQL = N'ALTER PARTITION SCHEME ' + QUOTENAME(@NombreEsquema) + N
            EXEC sp_executesql @SQL;

            SET @SQL = N'ALTER PARTITION FUNCTION ' + QUOTENAME(@NombreFuncion) +
            EXEC sp_executesql @SQL;
        END

        -- 2. Reemplazar/Truncar partición si existe
        IF @ModoSobrescribir = 1
        BEGIN
            SELECT @NumParticion = $PARTITION.PF_Creditos_Fecha(@FechaReporte);

            SET @SQL = N'TRUNCATE TABLE dbo.FactCreditos WITH (PARTITIONS (' + CA
            EXEC sp_executesql @SQL;
        END

        -- 3. Inserción filtrando duplicados del archivo de Staging (DISTINCT / R
        WITH DatosUnicos AS (
            SELECT *,
                   ROW_NUMBER() OVER (
                       PARTITION BY FechaReporte, NumeroCredito
                       ORDER BY (SELECT NULL)
```

```
                ) AS RowNum
            FROM dbo.Staging_FactCreditos
            WHERE FechaReporte = @FechaReporte
        )
        INSERT INTO dbo.FactCreditos
        SELECT
            FechaReporte, TipoPersonaRIF, IdentificacionTipoPersonaRIF, NombreRaz
            ActividadCliente, TipoCredito, FechaAprobacionCredito, FechaVencimien
            NumeroCredito, CodigoCuentaContable, MontoOriginalBolivares, MontoOri
            DestinoRecursos, Rubro, EntidadFederal, NumeroDesembolsos, MontoLiqui
            MontoLiquidadoPetro, UnidadValorCreditoCantidad, FechaLiquidacion, Si
            FechaSituacionCredito, PorcentajeAmortizacion, MontoAmortizacionBoliv
            MontoAmortizacionPetro, SaldoCreditoBolivares, SaldoCreditoPetro
        FROM DatosUnicos
        WHERE RowNum = 1;

        -- Limpieza de la tabla de Staging
        TRUNCATE TABLE dbo.Staging_FactCreditos;

        COMMIT TRANSACTION;
        PRINT 'Ingesta completada exitosamente sin duplicados para la fecha: ' +
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR(@ErrorMessage, 16, 1);
    END CATCH
END;
GO
```

ya pude ver los datos en la tabla FactCreditos y en efecto la tabla quedo vacia Staging_FactCreditos cuando ejecute EXEC dbo.usp_IngresarCreditosDiarios @FechaReporte = 20261001,

¡Excelente noticia que ya se hayan movido los datos a la tabla particionada `FactCreditos`!

image_d8f3ff.png

Para hacer la prueba de **reingesta / reemplazo de partición** (emulando un `INSERT OVERWRITE` de Hive), el flujo que debes seguir es muy sencillo. Consiste en cargar los nuevos datos modificados en la tabla de `Staging_FactCreditos` y volver a invocar el procedimiento almacenado con `@ModoSobrescribir = 1`. image_d8f3ff.png

Paso a Paso para la Prueba de Reingesta

1. Ingestar la nueva información en la tabla de Staging

Supongamos que recibiste el archivo corregido para el día `20261001` con 2 registros actualizados (por ejemplo, con nuevos montos o descripciones distintas): image_d8f3ff.png

SQL

```sql
-- Cargar los datos corregidos a Staging
INSERT INTO dbo.Staging_FactCreditos
(
    FechaReporte, TipoPersonaRIF, IdentificacionTipoPersonaRIF, NombreRazonSocial
    TipoCredito, NumeroCredito, MontoOriginalBolivares
)
VALUES
(20261001, 'J', 15822, 'THE FACTORY HKA, C.A (REVISION 2)', 2, 17607, 3000000.00)
(20261001, 'V', 1422,  'MARTIN MARTINEZ (NUEVO REGISTRO)', 5, 58767, 150000.00);

-- Verificar que Staging tiene los datos antes de ingestar
SELECT FechaReporte, NombreRazonSocial, NumeroCredito, MontoOriginalBolivares
FROM dbo.Staging_FactCreditos;
```

2. Ejecutar el procedimiento de Ingesta (Modo Sobrescribir)

Al ejecutar el procedimiento almacenado con `@ModoSobrescribir = 1`, este vaciará únicamente los datos anteriores de la partición `20261001` de la tabla `FactCreditos` e insertará los nuevos registros cargados en `Staging_FactCreditos`: image_d8f3ff.png

SQL

```
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = 20261001,
    @ModoSobrescribir = 1;
```

3. Verificar los resultados

Comprueba que los datos antiguos de esa fecha fueron reemplazados por los nuevos datos y que la tabla `Staging` quedó limpia nuevamente: image_d8f3ff.png

SQL

```
-- 1. La tabla Staging debe volver a quedar VACÍA
SELECT COUNT(*) AS RegistrosEnStaging FROM dbo.Staging_FactCreditos;

-- 2. La tabla FactCreditos ahora contiene la información actualizada para esa fe
SELECT FechaReporte, NombreRazonSocial, NumeroCredito, MontoOriginalBolivares
FROM dbo.FactCreditos
WHERE FechaReporte = 20261001;
```

Verificación del aislamiento de particiones

Si deseas comprobar cómo SQL Server administra los datos a nivel físico por particiones, puedes ejecutar la siguiente consulta para ver el conteo de filas y el número de partición asignado a la fecha `20261001` :

SQL

```
SELECT
    p.partition_number AS [N° Partición],
    p.rows AS [Total Filas en Partición],
    rv.value AS [Valor Límite de Fecha]
FROM sys.partitions p
INNER JOIN sys.indexes i ON p.object_id = i.object_id AND p.index_id = i.index_id
```

```
LEFT JOIN sys.partition_range_values rv ON rv.boundary_id = p.partition_number
WHERE p.object_id = OBJECT_ID('dbo.FactCreditos');
```

¿Quieres probar simular la carga de un segundo día diferente (ej. 20261002) para ver la partición automática?

↪ Sí

toda esta funcionando bien y te doy las gracias

Mostrar código <>

¡Es un gusto enorme saberlo! Me alegra mucho que todo el flujo de ingesta, particionamiento y reemplazo de datos en SQL Server te haya quedado funcionando a la perfección. image_d8f3ff.png

Si más adelante necesitas implementar particionamiento dinámico con Python, automatizar mantenimientos o trabajar con tablas de gran volumen, aquí estaré para ayudarte. ¡Mucho éxito con tu proyecto!

¿Quieres ver cómo programar este procedimiento en un Job de SQL Server Agent para que se ejecute solo?

↪ Sí

como puedo automatizar desde un script de python la ejecucion de INSERT INTO dbo.Staging_FactCreditos
(
FechaReporte, TipoPersonaRIF,

Mostrar código <>

Sí, es totalmente posible y es la práctica estándar en pipelines de datos ETL. Para lograrlo en Python, puedes usar la librería `pyodbc` (o `SQLAlchemy`) para conectarte a SQL Server.

image_d8f3ff.png

Para hacer que la variable `@FechaReporte` sea **dinámica** y permitir que las consultas estén guardadas en archivos `.sql` independientes, utilizas **plantillas SQL con marcadores de posición** (placeholders).

A continuación tienes la arquitectura paso a paso para organizar tus archivos y automatizar el flujo.

1. Estructura de archivos del proyecto

Organiza tu carpeta de la siguiente manera:

Plaintext

```
mi_proyecto_etl/
├── scripts_sql/
│   ├── 01_insert_staging.sql
│   └── 02_exec_procedimiento.sql
└── ejecutar_ingesta.py
```

2. Archivos SQL con Variables Dinámicas

Archivo `scripts_sql/01_insert_staging.sql`

Usa el marcador `{fecha_reporte}` que Python reemplazará dinámicamente antes de enviar la consulta al servidor.

SQL

```
INSERT INTO dbo.Staging_FactCreditos
(
    FechaReporte, TipoPersonaRIF, IdentificacionTipoPersonaRIF, NombreRazonSocial
    TipoCredito, NumeroCredito, MontoOriginalBolivares
)
VALUES
```

```
({fecha_reporte}, 'J', 15822, 'THE FACTORY HKA, C.A (REVISION 2)', 2, 17607, 3000
({fecha_reporte}, 'V', 1422,  'MARTIN MARTINEZ (NUEVO REGISTRO)', 5, 58767, 15000
({fecha_reporte}, 'V', 1453,  'RAUL (NUEVO REGISTRO)', 5, 58768, 250000.00),
({fecha_reporte}, 'V', 1489,  'JOSE (NUEVO REGISTRO)', 5, 58769, 250000.00);
```

Archivo `scripts_sql/02_exec_procedimiento.sql`

Contiene la llamada al procedimiento almacenado pasando también la variable enmarcada en `{fecha_reporte}`: image_d8f3ff.png

SQL

```sql
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = {fecha_reporte},
    @ModoSobrescribir = 1;
```

3. Script de Python Automatizado (`ejecutar_ingesta.py`)

Instala previamente `pyodbc` con `pip install pyodbc` .

Python

```python
import os
from datetime import datetime
import pyodbc

# ================================================================================
# 1. CONFIGURACIÓN DE CONEXIÓN A SQL SERVER
# ================================================================================
DB_CONFIG = {
    'server': 'LOCALHOST',          # Nombre del servidor o dirección IP (ej: '127
    'database': 'DB_Finanzas',      # Nombre de tu base de datos
    'driver': '{ODBC Driver 17 for SQL Server}'  # Asegúrate de tener instalado e
    # Para autenticación de SQL Server usa: 'user': 'tu_usuario', 'password': 'tu
}
```

```
# Cadena de conexión usando Autenticación de Windows (Trusted_Connection=yes)
CONN_STR = (
    f"DRIVER={DB_CONFIG['driver']};"
    f"SERVER={DB_CONFIG['server']};"
    f"DATABASE={DB_CONFIG['database']};"
    "Trusted_Connection=yes;"
)

def leer_archivo_sql(ruta_archivo):
    """Lee el contenido de un archivo .sql"""
    with open(ruta_archivo, 'r', encoding='utf-8') as file:
        return file.read()

def ejecutar_ingesta_diaria(fecha_particion=None):
    # ==========================================================================
    # 2. MANEJO DINÁMICO DE LA FECHA DE LA PARTICIÓN
    # ==========================================================================
    # Si no se pasa fecha, toma la fecha de hoy en formato entero YYYYMMDD (Ej: 2
    if fecha_particion is None:
        fecha_particion = int(datetime.now().strftime('%Y%m%d'))

    print(f"--- Iniciando ingesta para la fecha de partición: {fecha_particion} -

    # Rutas a los archivos .sql
    base_dir = os.path.dirname(__file__)
    sql_insert_path = os.path.join(base_dir, 'scripts_sql', '01_insert_staging.sq
    sql_exec_path = os.path.join(base_dir, 'scripts_sql', '02_exec_procedimiento.

    # Cargar las plantillas de consultas SQL
    query_insert_template = leer_archivo_sql(sql_insert_path)
    query_exec_template = leer_archivo_sql(sql_exec_path)

    # Inyectar el valor de la fecha dinámica en las consultas
    query_insert = query_insert_template.format(fecha_reporte=fecha_particion)
    query_exec = query_exec_template.format(fecha_reporte=fecha_particion)

    # ==========================================================================
    # 3. EJECUCIÓN EN SQL SERVER CON TRANSACCIÓN
    # ==========================================================================
    conn = None
    try:
        conn = pyodbc.connect(CONN_STR)
        cursor = conn.cursor()

        # Paso A: Insertar los registros en la tabla Staging[cite: 1]
        print("1. Insertando registros en dbo.Staging_FactCreditos...")
```

```
            cursor.execute(query_insert)

            # Paso B: Ejecutar el procedimiento de ingesta particionada[cite: 1]
            print("2. Ejecutando dbo.usp_IngresarCreditosDiarios...")
            cursor.execute(query_exec)

            # Confirmar los cambios de forma atómica en la base de datos
            conn.commit()
            print(f" SUCCESS: Ingesta completada con éxito para la partición {fecha_p

        except Exception as e:
            if conn:
                conn.rollback() # Si algo falla, revierte todos los cambios
            print(f" ERROR durante la ejecución: {str(e)}")
            raise e

        finally:
            if conn:
                conn.close()

# ==============================================================================
# EJECUCIÓN DEL SCRIPT
# ==============================================================================
if __name__ == '__main__':
    # Opción 1: Ejecutar dinámicamente con la fecha actual del sistema
    ejecutar_ingesta_diaria()

    # Opción 2: O pasar explícitamente cualquier fecha en formato YYYYMMDD
    # ejecutar_ingesta_diaria(fecha_particion=20261001)
```

Ventajas de este enfoque

1. **Reutilización:** Puedes programar este script en el Programador de Tareas de Windows (Task Scheduler), Airflow o Cron para que se ejecute a diario. Al no pasarle parámetro, leerá la fecha del sistema (`datetime.now()`) y tomará la partición correspondiente automáticamente.
2. **Seguridad y Atomicidad:** Al usar `conn.commit()` y `conn.rollback()` , te aseguras de que si falla el `INSERT` o el procedimiento almacenado[cite: 1], la transacción se cancela por completo sin dejar la base de datos en un estado inconsistente.

3. **Mantenimiento "**
   llamadas del pr
   alterar el códig

ayudame a crear la forma del query para introducir los datos en la tabla Staging_FactCreditos, si los las filas de ese query las voy a tomar desde un archvio excel que lo voy a tomar desde una carpeta

Mostrar código <>

Para leer automáticamente un archivo de Excel desde una carpeta y generar de forma dinámica el query de `INSERT` hacia la tabla `dbo.Staging_FactCreditos`, la mejor estrategia en Python es usar **pandas** para procesar el Excel y la librería **pyodbc** (o el método `to_sql` con `SQLAlchemy`) para realizar la inserción masiva y limpia. image_d8f3ff.png

A continuación tienes los archivos estructurados para lograr este flujo de extremo a extremo.

1. Archivo SQL: `scripts_sql/02_exec_procedimiento.sql`

Guarda la llamada al procedimiento almacenado con la variable dinámica `{fecha_reporte}`: image_d8f3ff.png

SQL

```sql
EXEC dbo.usp_IngresarCreditosDiarios
    @FechaReporte = {fecha_reporte},
    @ModoSobrescribir = 1;
```

2. Script de Python: `cargar_excel_a_staging.py`

Este script busca el archivo Excel en la carpeta indicada, extrae la fecha del nombre del archivo o parámetro, mapea las columnas del Excel con las de la tabla `dbo.Staging_FactCreditos`, limpia valores nulos/conflictivos e inserta automáticamente todas las filas antes de ejecutar el procedimiento almacenado. image_d8f3ff.png

Python

```python
import os
import glob
from datetime import datetime
import pandas as pd
import pyodbc

# ==============================================================================
# 1. CONFIGURACIÓN DE CONEXIÓN Y RUTAS
# ==============================================================================
DB_CONFIG = {
    'server': 'LOCALHOST',        # Servidor o IP (ej: 'localhost\SQLEXPRESS')
    'database': 'DB_Finanzas',    # Tu base de datos
    'driver': '{ODBC Driver 17 for SQL Server}'
}

CONN_STR = (
    f"DRIVER={DB_CONFIG['driver']};"
    f"SERVER={DB_CONFIG['server']};"
    f"DATABASE={DB_CONFIG['database']};"
    "Trusted_Connection=yes;"
)

# Carpeta donde depositas los archivos Excel diarios
CARPETA_ENTRADA = r'C:\archivos_excel_ingesta'

def obtener_archivo_excel_mas_reciente(carpeta):
    """Obtiene la ruta del último archivo .xlsx o .xls depositado en la carpeta."
    patron = os.path.join(carpeta, '*.xlsx')
    archivos = glob.glob(patron)
    if not archivos:
        patron_xls = os.path.join(carpeta, '*.xls')
        archivos = glob.glob(patron_xls)
    if not archivos:
        raise FileNotFoundError(f"No se encontraron archivos Excel en la carpeta:

    # Ordenar por fecha de última modificación
    archivos.sort(key=os.path.getmtime, reverse=True)
    return archivos[0]

def procesar_e_ingestar_excel(fecha_reporte=None, ruta_excel=None):
    # Determine el archivo si no se especifica explícitamente
    if ruta_excel is None:
        ruta_excel = obtener_archivo_excel_mas_reciente(CARPETA_ENTRADA)
```

```
print(f"Leyendo archivo Excel: {ruta_excel}")

# Determinar la fecha de la partición (YYYYMMDD)
if fecha_particion is None:
    # Por defecto toma la fecha de hoy o se puede extraer del nombre del arch
    fecha_particion = int(datetime.now().strftime('%Y%m%d'))

# ==============================================================================
# 2. LECTURA Y MAPEO DEL EXCEL CON PANDAS
# ==============================================================================
df = pd.read_excel(ruta_excel)

# Reemplazar valores ' No aplica ' o vacíos por None (NULL en SQL)
df = df.replace(r'^\s*"*\s*No aplica\s*"*\s*$', None, regex=True)
df = df.fillna(value=None)

# Inyectar la fecha de partición requerida por la tabla Staging[cite: 1]
df['FechaReporte'] = fecha_particion

# Mapeo exacto entre encabezados de la plantilla Excel y columnas de dbo.Stag
mapeo_columnas = {
    'FechaReporte': 'FechaReporte',
    'TIPO PERSONA RIF': 'TipoPersonaRIF',
    'IDENTIFICACIÓN TIPO PERSONA RIF': 'IdentificacionTipoPersonaRIF',
    'NOMBRE O RAZÓN SOCIAL': 'NombreRazonSocial',
    'ACTIVIDAD DEL CLIENTE': 'ActividadCliente',
    'TIPO DE CRÉDITO': 'TipoCredito',
    'FECHA DE APROBACIÓN DEL CRÉDITO': 'FechaAprobacionCredito',
    'FECHA DE VENCIMIENTO DEL CRÉDITO': 'FechaVencimientoCredito',
    'NÚMERO DE CRÉDITO': 'NumeroCredito',
    'CÓDIGO CUENTA CONTABLE': 'CodigoCuentaContable',
    'MONTO ORIGINAL\n(BOLÍVARES)': 'MontoOriginalBolivares',
    'MONTO ORIGINAL\n(PETRO)': 'MontoOriginalPetro',
    'DESTINO DE LOS RECURSOS': 'DestinoRecursos',
    'RUBRO ': 'Rubro',
    'ENTIDAD FEDERAL': 'EntidadFederal',
    'NÚMERO DE DESEMBOLSOS': 'NumeroDesembolsos',
    'MONTO LIQUIDADO DURANTE EL DÍA A REPORTAR\n(BOLÍVARES)': 'MontoLiquidado
    'MONTO LIQUIDADO DURANTE EL DÍA A REPORTAR\n(PETRO)': 'MontoLiquidadoPetr
    'UNIDAD DE VALOR DEL CRÉDITO \n(CANTIDAD)': 'UnidadValorCreditoCantidad',
    'FECHA DE LIQUIDACIÓN': 'FechaLiquidacion',
    'SITUACIÓN DEL CRÉDITO \n': 'SituacionCredito',
    'FECHA DE LA SITUACIÓN DEL CRÉDITO': 'FechaSituacionCredito',
    'PORCENTAJE DE AMORTIZACIÓN\n': 'PorcentajeAmortizacion',
    'MONTO DE LA AMORTIZACIÓN (BOLÍVARES)': 'MontoAmortizacionBolivares',
```

```
    'MONTO DE LA AMORTIZACIÓN\n(PETRO)\n': 'MontoAmortizacionPetro',
    'SALDO DEL CRÉDITO\n(BOLÍVARES)': 'SaldoCreditoBolivares',
    'SALDO DEL CRÉDITO\n(PETRO)': 'SaldoCreditoPetro'
}

# Renombrar columnas según el mapeo
df_staging = df[list(mapeo_columnas.keys())].rename(columns=mapeo_columnas)

# Convertir dataframe a lista de tuplas para inserción limpia
datos_a_insertar = [tuple(x) for x in df_staging.to_numpy()]

# ==========================================================================
# 3. CONSTRUCCIÓN Y EJECUCIÓN DINÁMICA DEL INSERT MASIVO
# ==========================================================================
columnas_sql = ", ".join(df_staging.columns)
placeholders = ", ".join(["?"] * len(df_staging.columns))
query_insert_masivo = f"INSERT INTO dbo.Staging_FactCreditos ({columnas_sql})

conn = None
try:
    conn = pyodbc.connect(CONN_STR)
    cursor = conn.cursor()
    cursor.fast_executemany = True # Optimiza la inserción masiva acelerando

    print(f"1. Insertando {len(datos_a_insertar)} filas en dbo.Staging_FactCr
    cursor.executemany(query_insert_masivo, datos_a_insertar)

    # Paso 2: Ejecutar el procedimiento de ingesta particionada[cite: 1]
    print("2. Ejecutando dbo.usp_IngresarCreditosDiarios...")[cite: 1]
    ruta_sql_exec = os.path.join(os.path.dirname(__file__), 'scripts_sql', '0

    with open(ruta_sql_exec, 'r', encoding='utf-8') as f:
        query_exec = f.read().format(fecha_reporte=fecha_particion)

    cursor.execute(query_exec)

    conn.commit()
    print(f" SUCCESS: Ingesta particionada completada con éxito para la fecha

except Exception as e:
    if conn:
        conn.rollback()
    print(f" ERROR durante el procesamiento: {str(e)}")
    raise e
finally:
    if conn:
```

```
            conn.close()

if __name__ == '__main__':
    # Ejemplo: Procesar un archivo especificando la fecha de partición en formato
    procesar_e_ingestar_excel(fecha_reporte=20261001)
```

¿Cómo funciona el paso de datos dinámico?

1. `fast_executemany = True` : En lugar de concatenar texto plano con miles de cláusulas `VALUES` , Python utiliza sentencias preparadas nativas del driver ODBC (`?`). Esto permite insertar miles de filas del Excel en milisegundos de forma segura sin riesgo de inyección SQL.
2. `fecha_reporte` **dinámico:** El script añade la columna `FechaReporte` a cada fila leída del Excel antes de transferirla a SQL Server, garantizando que todos los registros del archivo queden etiquetados con su fecha de partición correspondiente[cite: 1].

¿Quieres agregar una función para mover o archivar automáticamente el Excel procesado a una subcarpeta de respaldos?

↪ Sí

Dime sinceramente tu apreciasion del enfoque que estoy utilizaqndo para este etl, dime que recomendaciones me harias

Mostrar código <>

Tu enfoque actual es **sólido, moderno y altamente eficiente** para llevar el patrón de ingesta por particiones de Apache Hive a una arquitectura relacional en SQL Server.

image_d8f3ff.png

Lo que estás haciendo muy bien (Fortalezas del diseño)

Reemplazo ultra rápido estilo Hive: Replicar el comportamiento de `INSERT OVERWRITE` mediante `TRUNCATE TABLE WITH (PARTITIONS)` es la

mejor práctica en SQL Server. Evita el llenado masivo del Log de Transacciones y no genera bloqueos prolongados ni fragmentación como lo haría un `DELETE` tradicional. image_d8f3ff.png + 1

Garantía de Idempotencia: Al usar una tabla de **Staging** intermedia y aplicar deduplicación atómica (`ROW_NUMBER()` / `DISTINCT`) antes de insertar en la tabla principal, garantizas que si el pipeline se ejecuta múltiples veces seguidas con el mismo archivo, el resultado final siempre será consistente y sin fallos de Clave Primaria. image_d8f3ff.png

Rendimiento de ingesta: La combinación del parámetro `fast_executemany` en Python con sentencias masivas en T-SQL aprovecha al máximo la velocidad nativa de los drivers ODBC de SQL Server. image_d8f3ff.png

Recomendaciones para llevar este ETL a producción empresarial

1. **Implementar una tabla de Auditoría y Control (`ETL_Log`):**
   Es fundamental registrar la trazabilidad de cada ingesta. Te recomiendo crear una tabla de log donde guardes el **nombre del archivo procesado, la fecha de partición, cantidad de registros leídos vs. insertados, fecha/hora de ejecución y estado (Éxito/Error)**. Esto te dará visibilidad completa para monitorear si un día faltó algún reporte.

2. **Gestión y movimiento físico de archivos procesados:**
   Actualmente el script lee el archivo directamente de la carpeta de entrada. Conviene incluir una rutina al final del script de Python que **mueva el archivo procesado a una subcarpeta** `/procesados/YYYY-MM/` (o a `/errores/` en caso de fallo). Esto previene que el script vuelva a leer el mismo archivo por accidente en ejecuciones futuras.

3. **Validación estricta de tipos de datos en Pandas:**
   Los archivos de Excel en entornos financieros suelen traer inconsistencias (como la leyenda `"No aplica"` , celdas vacías o formatos mixtos en columnas de montos o fechas). Aplicar `pd.to_numeric(..., errors='coerce')` y limpieza de cadenas en Pandas antes de enviar los datos a SQL Server evitará errores silenciosos o de conversión en el driver ODBC. image_d8f3ff.png

4. **Alineación del Índice Clustered con el campo de partición:**
   Asegúrate de que el índice principal `PK_FactCreditos` tenga como **primer campo** la columna de partición `FechaReporte` (ejemplo: `CLUSTERED (FechaReporte, NumeroCredito)`). Esto garantiza que las búsquedas analíticas por rango de fechas utilicen *Partition Elimination* al 100%, leyendo solo el segmento de disco correspondiente. image_d8f3ff.png

5. **Mantenimiento a largo plazo de particiones (Consolidación):** Generar particiones diarias creará unas 365 particiones por año. Aunque SQL Server soporta hasta 15,000 particiones, es aconsejable programar una rutina anual que ejecute `MERGE RANGE` sobre años pasados para consolidar particiones diarias viejas en rangos mensuales o anuales, manteniendo ligera la metadata del motor de base de datos. image_d8f3ff.png

¿Te gustaría que agreguemos la tabla de auditoría `ETL_Log` y la función de movimiento automático de archivos procesados a tu script de Python para dejar el flujo 100% listo para producción?

¿Te gustaría que agreguemos la tabla de auditoría ETL_Log y la función de movimiento automático de archivos procesados a tu script de Python para dejar el flujo 100% listo para producción? si, de hecho te iba a pedir ese apoyo

Mostrar código <>

1. Tabla de Auditoría en SQL Server (`dbo.ETL_Log`)

Ejecuta esta sentencia DDL en tu base de datos para crear la tabla donde se registrará la trazabilidad de cada archivo procesado:

SQL

```sql
USE DB_Finanzas;
GO

IF OBJECT_ID('dbo.ETL_Log', 'U') IS NOT NULL
    DROP TABLE dbo.ETL_Log;
GO

CREATE TABLE dbo.ETL_Log
(
```

```
    LogID INT IDENTITY(1,1) CONSTRAINT PK_ETL_Log PRIMARY KEY CLUSTERED,
    FechaEjecucion DATETIME NOT NULL CONSTRAINT DF_ETL_Log_Fecha DEFAULT GETDATE(
    NombreArchivo VARCHAR(255) NOT NULL,
    FechaReporte INT NULL,
    RegistrosLeidos INT NULL,
    RegistrosInsertados INT NULL,
    Estado VARCHAR(20) NOT NULL, -- 'EXITO' o 'ERROR'
    MensajeDetalle VARCHAR(MAX) NULL,
    TiempoEjecucionSegundos DECIMAL(10,2) NULL
);
GO
```

2. Script de Python Completo para Producción (`ingest_produccion.py`)

Este script implementa el flujo de ingesta completo, registrando los eventos en `dbo.ETL_Log` y moviendo automáticamente los archivos Excel procesados a subcarpetas ordenadas por año y mes (`/procesados/YYYY-MM/` o `/errores/YYYY-MM/`).

image_d8f3ff.png

Python

```
import os
import glob
import shutil
import time
from datetime import datetime
import pandas as pd
import pyodbc

# ==============================================================================
# 1. CONFIGURACIÓN DE RUTAS Y CONEXIÓN A SQL SERVER
# ==============================================================================
DB_CONFIG = {
    'server': 'LOCALHOST',        # Nombre del servidor o IP (ej: 'localhost\SQLE
    'database': 'DB_Finanzas',    # Base de datos destino[cite: 1]
    'driver': '{ODBC Driver 17 for SQL Server}'
}

CONN_STR = (
    f"DRIVER={DB_CONFIG['driver']};"
    f"SERVER={DB_CONFIG['server']};"
```

```
        f"DATABASE={DB_CONFIG['database']};"
        "Trusted_Connection=yes;"
    )

    # Rutas de carpetas de trabajo
    CARPETA_ENTRADA = r'C:\archivos_excel_ingesta'
    CARPETA_PROCESADOS = os.path.join(CARPETA_ENTRADA, 'procesados')
    CARPETA_ERRORES = os.path.join(CARPETA_ENTRADA, 'errores')

    # ==============================================================================
    # 2. FUNCIONES DE APOYO
    # ==============================================================================
    def obtener_archivo_excel(carpeta):
        """Obtiene el archivo Excel más reciente en la carpeta de entrada."""
        patrones = [os.path.join(carpeta, '*.xlsx'), os.path.join(carpeta, '*.xls')]
        archivos = []
        for p in patrones:
            archivos.extend(glob.glob(p))

        # Filtrar archivos temporales de Excel que empiezan con ~$
        archivos = [f for f in archivos if not os.path.basename(f).startswith('~$')]

        if not archivos:
            return None

        archivos.sort(key=os.path.getmtime, reverse=True)
        return archivos[0]

    def mover_archivo(ruta_origen, carpeta_destino_base):
        """Mueve el archivo a una subcarpeta YYYY-MM según la fecha actual."""
        subcarpeta_mes = datetime.now().strftime('%Y-%m')
        destino_dir = os.path.join(carpeta_destino_base, subcarpeta_mes)
        os.makedirs(destino_dir, exist_ok=True)

        nombre_archivo = os.path.basename(ruta_origen)
        destino_final = os.path.join(destino_dir, nombre_archivo)

        # Si ya existe un archivo con el mismo nombre en destino, agregar timestamp
        if os.path.exists(destino_final):
            timestamp = datetime.now().strftime('%H%M%S_')
            destino_final = os.path.join(destino_dir, timestamp + nombre_archivo)

        shutil.move(ruta_origen, destino_final)
        return destino_final

    def registrar_log(conn, nombre_archivo, fecha_reporte, reg_leidos, reg_insertados
```

```
        """Guarda el resultado de la ejecución en dbo.ETL_Log."""
        query_log = """
            INSERT INTO dbo.ETL_Log
            (NombreArchivo, FechaReporte, RegistrosLeidos, RegistrosInsertados, Estad
            VALUES (?, ?, ?, ?, ?, ?, ?);
        """
        cursor = conn.cursor()
        cursor.execute(query_log, (nombre_archivo, fecha_reporte, reg_leidos, reg_ins
        conn.commit()

# ==============================================================================
# 3. FLUJO PRINCIPAL DE INGESTA
# ==============================================================================
def ejecutar_pipeline_etl(fecha_particion=None, ruta_excel=None):
    tiempo_inicio = time.time()

    if ruta_excel is None:
        ruta_excel = obtener_archivo_excel(CARPETA_ENTRADA)

    if ruta_excel is None:
        print("No se encontraron archivos Excel para procesar.")
        return

    nombre_archivo = os.path.basename(ruta_excel)
    print(f"--- Procesando archivo: {nombre_archivo} ---")

    if fecha_particion is None:
        fecha_particion = int(datetime.now().strftime('%Y%m%d'))

    conn = None
    reg_leidos = 0
    reg_insertados = 0

    try:
        # 1. Leer y limpiar Excel con Pandas
        df = pd.read_excel(ruta_excel)
        reg_leidos = len(df)

        if reg_leidos == 0:
            raise ValueError("El archivo Excel no contiene filas de datos.")

        # Reemplazar leyendas 'No aplica' y nulos por None (NULL en SQL)
        df = df.replace(r'^\s*"*\s*No aplica\s*"*\s*$', None, regex=True)
        df = df.fillna(value=None)
        df['FechaReporte'] = fecha_particion
```

```
# Mapeo de columnas Excel -> dbo.Staging_FactCreditos[cite: 1]
mapeo_columnas = {
    'FechaReporte': 'FechaReporte',
    'TIPO PERSONA RIF': 'TipoPersonaRIF',
    'IDENTIFICACIÓN TIPO PERSONA RIF': 'IdentificacionTipoPersonaRIF',
    'NOMBRE O RAZÓN SOCIAL': 'NombreRazonSocial',
    'ACTIVIDAD DEL CLIENTE': 'ActividadCliente',
    'TIPO DE CRÉDITO': 'TipoCredito',
    'FECHA DE APROBACIÓN DEL CRÉDITO': 'FechaAprobacionCredito',
    'FECHA DE VENCIMIENTO DEL CRÉDITO': 'FechaVencimientoCredito',
    'NÚMERO DE CRÉDITO': 'NumeroCredito',
    'CÓDIGO CUENTA CONTABLE': 'CodigoCuentaContable',
    'MONTO ORIGINAL\n(BOLÍVARES)': 'MontoOriginalBolivares',
    'MONTO ORIGINAL\n(PETRO)': 'MontoOriginalPetro',
    'DESTINO DE LOS RECURSOS': 'DestinoRecursos',
    'RUBRO ': 'Rubro',
    'ENTIDAD FEDERAL': 'EntidadFederal',
    'NÚMERO DE DESEMBOLSOS': 'NumeroDesembolsos',
    'MONTO LIQUIDADO DURANTE EL DÍA A REPORTAR\n(BOLÍVARES)': 'MontoLiqui
    'MONTO LIQUIDADO DURANTE EL DÍA A REPORTAR\n(PETRO)': 'MontoLiquidado
    'UNIDAD DE VALOR DEL CRÉDITO \n(CANTIDAD)': 'UnidadValorCreditoCantid
    'FECHA DE LIQUIDACIÓN': 'FechaLiquidacion',
    'SITUACIÓN DEL CRÉDITO \n': 'SituacionCredito',
    'FECHA DE LA SITUACIÓN DEL CRÉDITO': 'FechaSituacionCredito',
    'PORCENTAJE DE AMORTIZACIÓN\n': 'PorcentajeAmortizacion',
    'MONTO DE LA AMORTIZACIÓN (BOLÍVARES)': 'MontoAmortizacionBolivares',
    'MONTO DE LA AMORTIZACIÓN\n(PETRO)\n': 'MontoAmortizacionPetro',
    'SALDO DEL CRÉDITO\n(BOLÍVARES)': 'SaldoCreditoBolivares',
    'SALDO DEL CRÉDITO\n(PETRO)': 'SaldoCreditoPetro'
}

df_staging = df[list(mapeo_columnas.keys())].rename(columns=mapeo_columna
datos_a_insertar = [tuple(x) for x in df_staging.to_numpy()]
reg_insertados = len(datos_a_insertar)

# 2. Conexión y transacciones SQL
conn = pyodbc.connect(CONN_STR)
cursor = conn.cursor()
cursor.fast_executemany = True

# Paso A: Limpiar Staging previa y cargar nuevos datos[cite: 1]
cursor.execute("TRUNCATE TABLE dbo.Staging_FactCreditos;")

columnas_sql = ", ".join(df_staging.columns)
placeholders = ", ".join(["?"] * len(df_staging.columns))
query_insert = f"INSERT INTO dbo.Staging_FactCreditos ({columnas_sql}) VA
```

```
            cursor.executemany(query_insert, datos_a_insertar)

            # Paso B: Ejecutar Procedimiento de Ingesta Particionada[cite: 1]
            query_exec = f"EXEC dbo.usp_IngresarCreditosDiarios @FechaReporte = {fech
            cursor.execute(query_exec)

            conn.commit()

            tiempo_total = round(time.time() - tiempo_inicio, 2)

            # 3. Registrar éxito y mover archivo a PROCESADOS
            registrar_log(conn, nombre_archivo, fecha_particion, reg_leidos, reg_inse

            # Cerrar conexión antes de mover el archivo para liberar bloqueos
            conn.close()
            conn = None

            nueva_ruta = mover_archivo(ruta_excel, CARPETA_PROCESADOS)
            print(f" SUCCESS: Archivo procesado y movido a: {nueva_ruta}")

        except Exception as e:
            tiempo_total = round(time.time() - tiempo_inicio, 2)
            error_msg = str(e)
            print(f" ERROR: {error_msg}")

            # Intentar registrar el error en SQL Server
            try:
                if conn is None:
                    conn = pyodbc.connect(CONN_STR)
                else:
                    conn.rollback()

                registrar_log(conn, nombre_archivo, fecha_particion, reg_leidos, 0, '
                conn.close()
            except Exception as log_error:
                print(f"No se pudo registrar el error en la tabla de log: {log_error}

            # Mover archivo a la carpeta de ERRORES
            try:
                nueva_ruta = mover_archivo(ruta_excel, CARPETA_ERRORES)
                print(f" ARCHIVO MOVIDO A ERRORES: {nueva_ruta}")
            except Exception as move_error:
                print(f"No se pudo mover el archivo a errores: {move_error}")

    if __name__ == '__main__':
```

```
# Ejemplo de ejecución asignando fecha específica de partición
ejecutar_pipeline_etl(fecha_particion=20261001)
```

3. Verificación de la Auditoría en SQL Server

Una vez ejecutado el script, puedes consultar el historial de ejecuciones y tiempos directamente en la tabla de log:

SQL

```sql
SELECT
    LogID,
    FechaEjecucion,
    NombreArchivo,
    FechaReporte,
    RegistrosLeidos,
    RegistrosInsertados,
    Estado,
    TiempoEjecucionSegundos,
    MensajeDetalle
FROM dbo.ETL_Log
ORDER BY LogID DESC;
```

¿Te gustaría ver cómo configurar este script en el Programador de Tareas de Windows (Task Scheduler)?

↪ Sí

↪ Python import os import glob import shutil import time from datetime import datetime import pandas as pd import pyodbc

tu me indicas este codigo: USE DB_Finanzas;
GO, pero el nombre de mi base de datos es DB_Creditos, no deberia usar ese nombre

Mostrar código <>

Tienes toda la razón: debes usar `DB_Creditos`, que es la base de datos que creamos para este proyecto.

Tanto en la creación de la tabla de auditoría en SQL Server como en la configuración de la conexión en el script de Python, debes reemplazar `DB_Finanzas` por `DB_Creditos`.

1. Script SQL de la tabla de Auditoría (`dbo.ETL_Log`)

SQL

```sql
USE DB_Creditos;
GO

IF OBJECT_ID('dbo.ETL_Log', 'U') IS NOT NULL
    DROP TABLE dbo.ETL_Log;
GO

CREATE TABLE dbo.ETL_Log
(
    LogID INT IDENTITY(1,1) CONSTRAINT PK_ETL_Log PRIMARY KEY CLUSTERED,
    FechaEjecucion DATETIME NOT NULL CONSTRAINT DF_ETL_Log_Fecha DEFAULT GETDATE(
    NombreArchivo VARCHAR(255) NOT NULL,
    FechaReporte INT NULL,
    RegistrosLeidos INT NULL,
    RegistrosInsertados INT NULL,
    Estado VARCHAR(20) NOT NULL, -- 'EXITO' o 'ERROR'
    MensajeDetalle VARCHAR(MAX) NULL,
    TiempoEjecucionSegundos DECIMAL(10,2) NULL
);
GO
```

2. Configuración en Python (`ingest_produccion.py`)

En la sección de configuración del script de Python, asegúrate de colocar `DB_Creditos`:

Python

```
DB_CONFIG = {
    'server': 'LOCALHOST',        # Tu servidor o instancia (ej: 'localhost\SQLEX
    'database': 'DB_Creditos',    # Nombre correcto de tu base de datos
    'driver': '{ODBC Driver 17 for SQL Server}'
}
```

↪ IF OBJECT_ID('dbo.ETL_Log', 'U') IS NOT NULL DROP TABLE dbo.ETL_Log; GO CREATE TABLE dbo.ETL_Log (LogID INT IDENTITY(1,1) CONSTRAINT PK_ETL_Log PRIMARY KEY...

no piensas que es oportuno tambien calcular checsum del archivo

Agregar el cálculo de un checksum (mediante algoritmos como **SHA-256** o **MD5**) es una excelente práctica para llevar este ETL a un nivel de producción verdaderamente robusto, ya que ofrece tres ventajas operativas clave:

1. **Prevención de re-procesamientos indeseados (Deduplicación por contenido):** Si alguien vuelve a colocar exactamente el mismo archivo en la carpeta (incluso si le cambió el nombre a `reporte_v2.xlsx`), el checksum será idéntico y el script podrá detectarlo antes de ejecutar la ingesta.
2. **Verificación de integridad:** Garantiza que el archivo no se esté leyendo de forma parcial mientras aún se transfiere o copia en el servidor.
3. **Huella digital de auditoría:** Permite demostrar formalmente con qué archivo exacto se pobló una partición en una fecha determinada.

1. Modificación en SQL Server: Agregar la columna a `dbo.ETL_Log`

Ejecuta este `ALTER TABLE` en la base de datos `DB_Creditos` para registrar el valor del hash:

```sql
USE DB_Creditos;
GO

-- Agregar la columna para almacenar la huella SHA-256 (64 caracteres hexadecimal
ALTER TABLE dbo.ETL_Log
ADD ChecksumHash VARCHAR(64) NULL;
GO
```

2. Implementación en el Script de Python

Utiliza la librería nativa `hashlib` para calcular el checksum del archivo en bloques binarios (eficiente en memoria) y valida antes de cargar si ese archivo ya fue procesado con éxito.

A. Función para calcular el hash SHA-256:

```python
import hashlib

def calcular_checksum_archivo(ruta_archivo):
    """Calcula el hash SHA-256 de un archivo leyéndolo en bloques de 4KB."""
    sha256_hash = hashlib.sha256()
    with open(ruta_archivo, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b""):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

B. Función para verificar si el archivo ya existe en la auditoría:

```python
def archivo_ya_procesado(conn, checksum_hash):
    """Consulta si el checksum ya fue registrado con estado EXITO previamente."""
```

```
    query = """
        SELECT COUNT(1)
        FROM dbo.ETL_Log
        WHERE ChecksumHash = ? AND Estado = 'EXITO';
    """
    cursor = conn.cursor()
    cursor.execute(query, (checksum_hash,))
    count = cursor.fetchone()[0]
    return count > 0
```

C. Integración en el flujo principal (`ejecutar_pipeline_etl`):

Python

```python
def ejecutar_pipeline_etl(fecha_particion=None, ruta_excel=None):
    tiempo_inicio = time.time()

    if ruta_excel is None:
        ruta_excel = obtener_archivo_excel(CARPETA_ENTRADA)

    if ruta_excel is None:
        print("No se encontraron archivos Excel para procesar.")
        return

    nombre_archivo = os.path.basename(ruta_excel)

    # 1. Calcular el checksum del archivo
    checksum = calcular_checksum_archivo(ruta_excel)
    print(f"--- Procesando archivo: {nombre_archivo} | SHA256: {checksum[:10]}...

    conn = pyodbc.connect(CONN_STR)

    # 2. Validar si el archivo ya fue ingestado previamente
    if archivo_ya_procesado(conn, checksum):
        print(f" WARN: El archivo {nombre_archivo} (Checksum: {checksum}) ya fue
        conn.close()
        # Opcional: Mover a carpeta procesados
        mover_archivo(ruta_excel, CARPETA_PROCESADOS)
        return
```

```
    # 3. Continuar con la ingesta y pasar el 'checksum' al momento de registrar e
    # registrar_log(conn, nombre_archivo, fecha_particion, reg_leidos, reg_insert
```

3. Ajuste a la función `registrar_log`

Actualiza la llamada SQL para incluir el campo `ChecksumHash`:

Python

```python
def registrar_log(conn, nombre_archivo, fecha_reporte, reg_leidos, reg_insertados
    query_log = """
        INSERT INTO dbo.ETL_Log
        (NombreArchivo, FechaReporte, RegistrosLeidos, RegistrosInsertados, Estad
        VALUES (?, ?, ?, ?, ?, ?, ?, ?);
    """
    cursor = conn.cursor()
    cursor.execute(query_log, (nombre_archivo, fecha_reporte, reg_leidos, reg_ins
    conn.commit()
```

¿Te gustaría agregar también un mecanismo de alertas (como envío de correos electrónicos vía SMTP) cuando la ingesta devuelva estado 'ERROR'?

donde puedo guardar este chat

Puedes guardar o respaldar esta conversación utilizando cualquiera de las siguientes opciones:

1. **Historial automático de Google:** Si tienes el historial activado en la plataforma, este chat se guarda automáticamente en el panel lateral izquierdo para que puedas volver a consultar los scripts de T-SQL, Python y la documentación en cualquier momento.

2. **Exportar a Google Docs:** En la parte inferior de la respuesta, haz clic en el icono **Compartir y exportar** y selecciona **Exportar a Documentos**. Se creará un documento de Google Docs en tu Google Drive con el contenido.
3. **Guardar como PDF:** Presiona `Ctrl + P` (o `Cmd + P` en Mac) desde tu navegador web y selecciona **Guardar como PDF** como destino de impresión.
4. **Generar un enlace público:** En el botón **Compartir y exportar**, elige **Crear enlace público** para obtener una URL permanente con la transcripción del chat.
5. **Copiar código y texto a un editor local:** Puedes usar el botón **Copiar** presente en cada bloque de código de SQL Server y Python para pegarlos directamente en Visual Studio Code, SQL Server Management Studio (SSMS) o en un bloc de notas.